

ЛЕКЦИЯ 4

АДАПТИВНАЯ ВЕРСТКА
И КРОССБРАУЗЕРНОСТЬ

ВОПРОСЫ

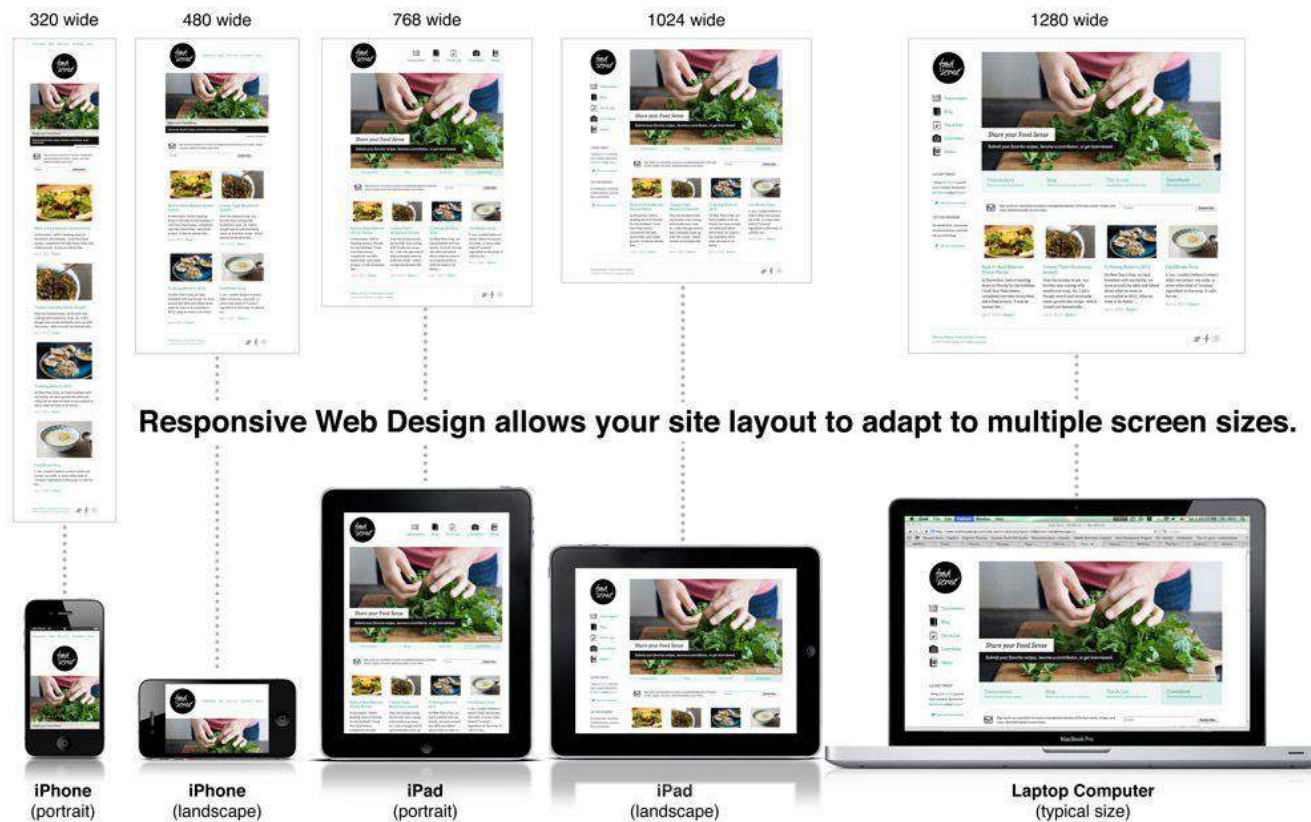
1. Адаптивный, отзывчивый или мобильный
2. Flexbox
3. Grid Layout
4. Masonry layout
5. @media/Медиазапросы

1

**АДАПТИВНЫЙ,
ОТЗЫВЧИВЫЙ
ИЛИ МОБИЛЬНЫЙ**

Когда вы открываете сайт или веб-приложение с телефона, то он выглядит иначе, чем на ноутбуке. Это происходит за счет продуманного дизайна.

Чтобы сайт выглядел хорошо и работал удобно на любых устройствах – от маленьких смартфонов до больших мониторов, – используются **специальные подходы к вёрстке**.



Адаптивная
верстка

Отзывчивый
дизайн

Мобильные
версии?

Подходы к верстке

Во главе веб-разработки в наше время стоит адаптивность – свойство, позволяющее сайту корректно отображаться на экранах любого размера: хоть на планшетах, хоть на смартфонах, проекторах, телевизорах.

Термин **«адаптивный»** означает дизайн, автоматически подстраивающийся под экран каждого устройства.

Сам термин стал известен благодаря книге «Отзывчивый веб-дизайн» за авторством Итана Маркотта.

АДАПТИВНЫЙ ДИЗАЙН САЙТА *на примере устройств с разным разрешением экрана*



Адаптивный дизайн

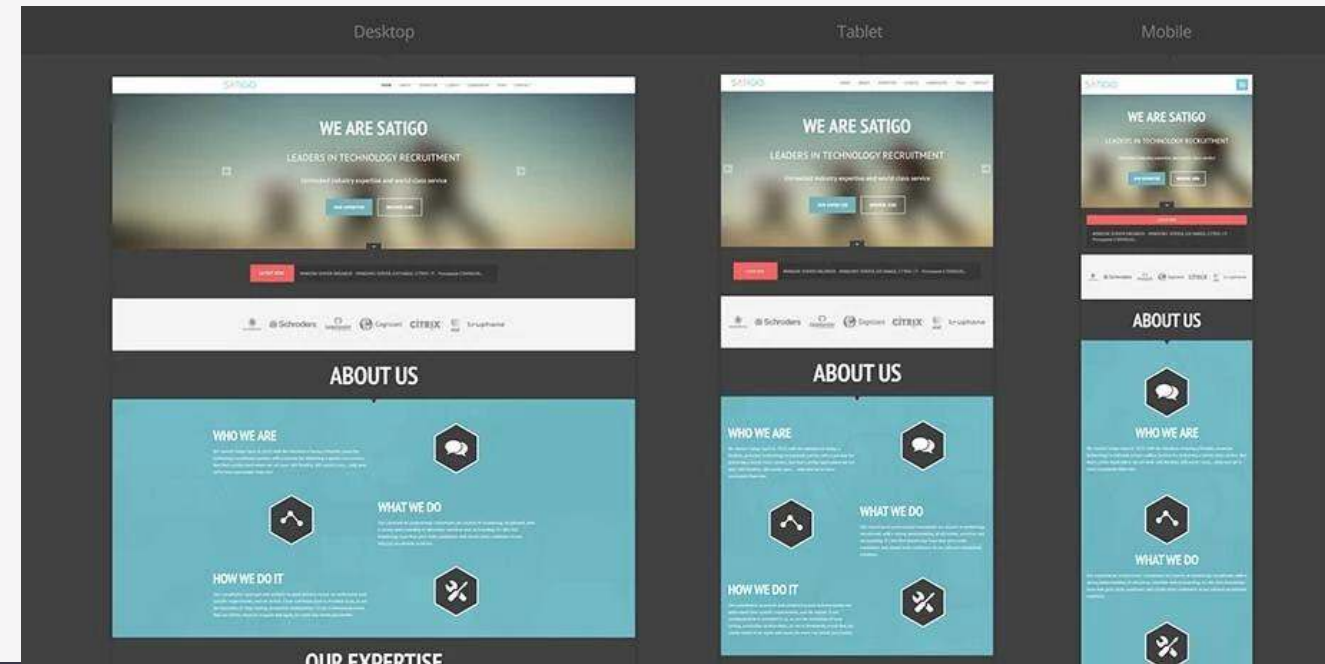
Сайт заранее настроен на несколько вариантов разрешений экрана и корректно отображается на различных устройствах.

Посмотрим на реальном примере, как выглядит адаптивный сайт на трех разных устройствах. Так адаптивный сайт отображается на разных устройствах: смартфонах (с горизонтальной и вертикальной ориентацией), десктопах и планшетах (также с горизонтальной и вертикальной ориентацией).

Что тут важно?

Во-первых, сайт удобен для посетителя при открытии с любого устройства.

Во-вторых, все элементы одинаково отображаются на экранах всех устройств. Мелкие элементы и шрифты остаются читаемыми.



Как отображается адаптивный сайт?

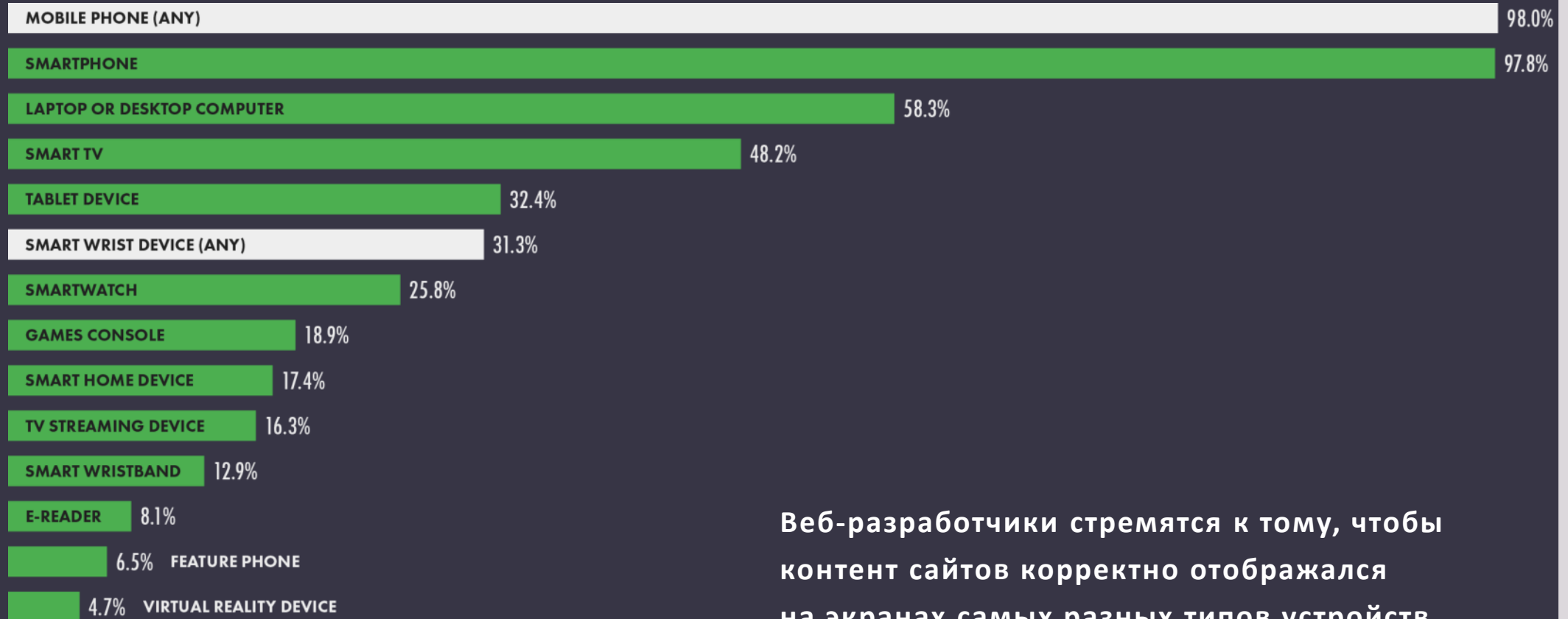
Устройства, с которых можно выходить в интернет, не ограничиваются десктопами и смартфонами. Необходимо учитывать, как сайт отображается и на других видах устройств, например, планшетах и умных телевизорах. Поправка на эти устройства появилась и во многих веб-аналитиках, в частности в Google Analytics и «Яндекс.Метрике».

И хотя посетители, просматривающие веб-страницы через телевизоры, все еще остаются диковинкой, в ближайшем будущем стоит ожидать увеличения такой доли устройств.

Метрики								Выберите цель			
Тип устройства, Производитель устройства, Модель устройства								Визиты	Посетители	Отказы	Глубина просмотра
Итого и средние								458	262	18,6%	2,83
ПК								262	122	18,7%	3,69
Смартфоны								185	129	17,8%	1,68
Apple								88	73	23,9%	1,42
Samsung								31	14	25,8%	2,06
HTC								11	1	9,09%	2,64
Beeline								8	3	12,5%	1,5
Xiaomi								8	5	0%	1,88
ТВ								4	4	75%	0

Зачем нужен адаптив?

Зачем нужен адаптив?



Веб-разработчики стремятся к тому, чтобы контент сайтов корректно отображался на экранах самых разных типов устройств.

Из-за разнообразия разрешений экранов и форматов процесс разработки значительно усложнился. Во многих случаях возникают проблемы при попытке переноса уже существующего сайта на адаптивный «шаблон». Особенно если сайт самописный и был создан много лет назад.

Удобство / юзабилити для мобильных устройств / адаптивность – факторы ранжирования, которые учитываются Google и «Яндекс». К ним можно отнести сразу несколько показателей:

1. скорость загрузки страниц на мобильных устройствах;
2. удобство навигации;
3. отсутствие слишком мелких элементов, которые нечитаемы с экранов мобильных устройств;
4. размер шрифта;
5. отсутствие элементов за пределами экрана.



Если сайт не адаптирован для мобильного трафика, то начнут расти отказы и ухудшаться поведенческие факторы аудитории. Что ведет к потере позиций в выдаче и снижению трафика.

Сложности адаптивной разработки

Аспект адаптивности учитывают не только веб-разработчики, но и дизайнеры, верстальщики, другие специалисты, которые занимаются созданием сайтов.

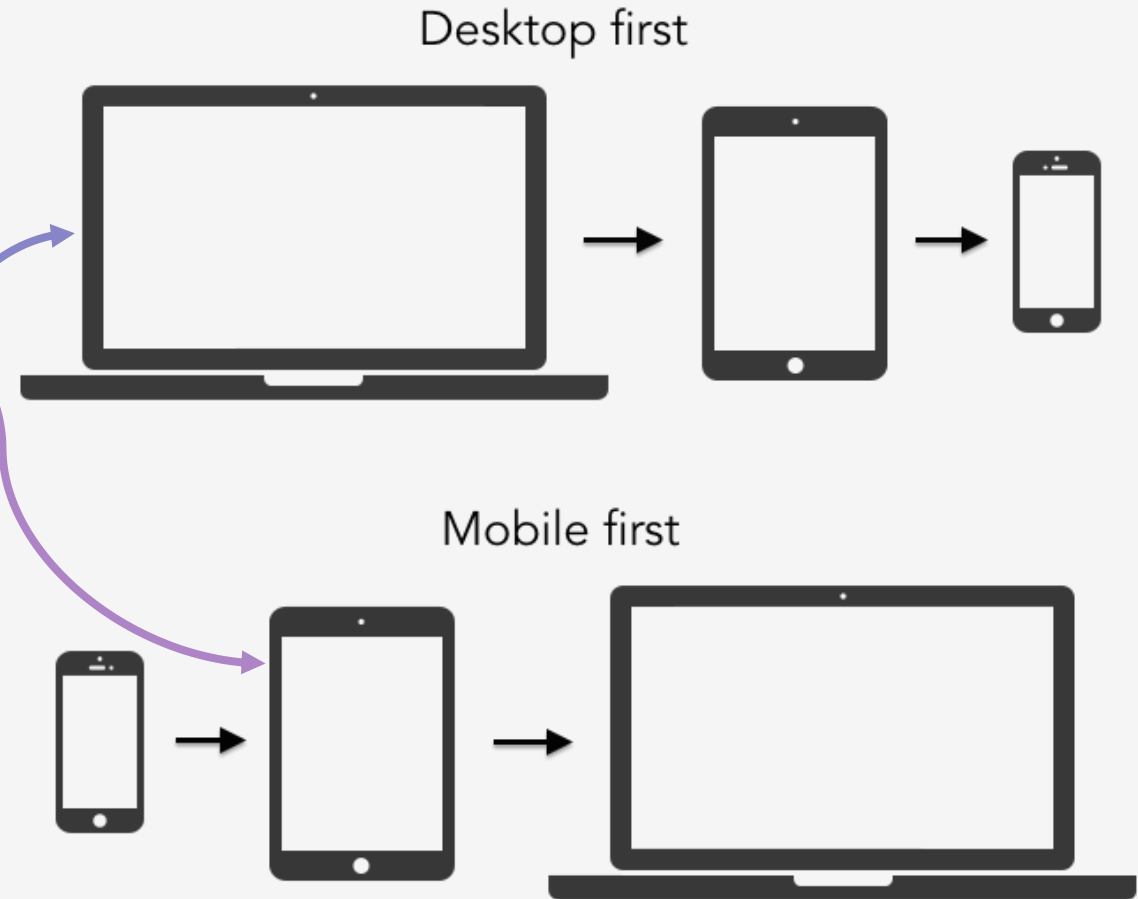
Какой экран верстать сначала?

Есть два основных подхода.

Первый: за основу берется экран смартфона, и затем работа по расположению элементов двигается в сторону увеличения размеров экрана.

Второй: за основу берется экран классического монитора, и далее прорабатывается отображение элементов на более мелких экранах.

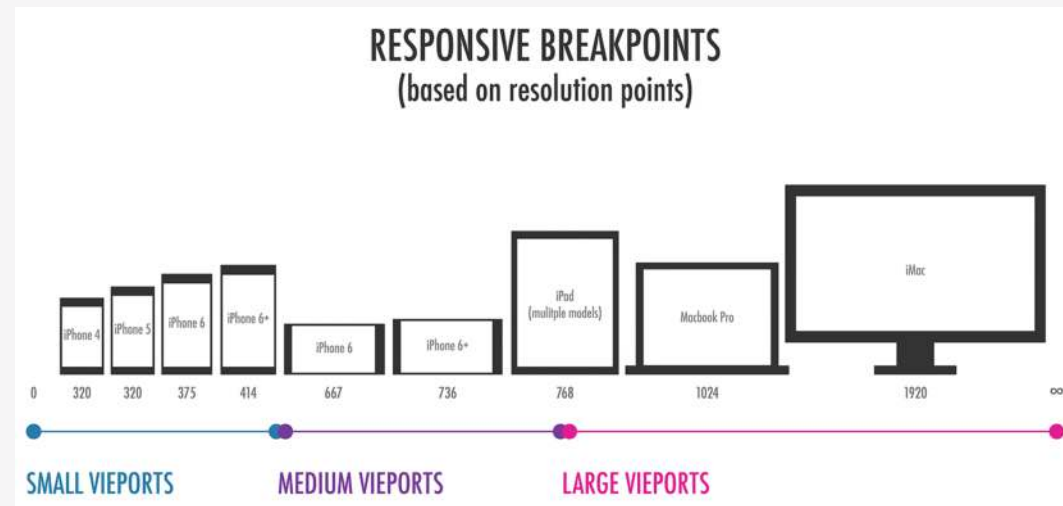
Раньше наиболее распространенным способом разработки был второй вариант. Сегодня более актуален первый способ – эта концепция получила название **Mobile First**. Этот способ разработки является более логичным и простым изначально.



Размеры макетов. Брейкпоинты

При создании адаптивного дизайна разработчик ориентируется не на конкретные значения, а на так называемые брейкпоинты. Брейкпоинты указывают конкретное разрешение экрана, при котором должно произойти изменение дизайна страницы.

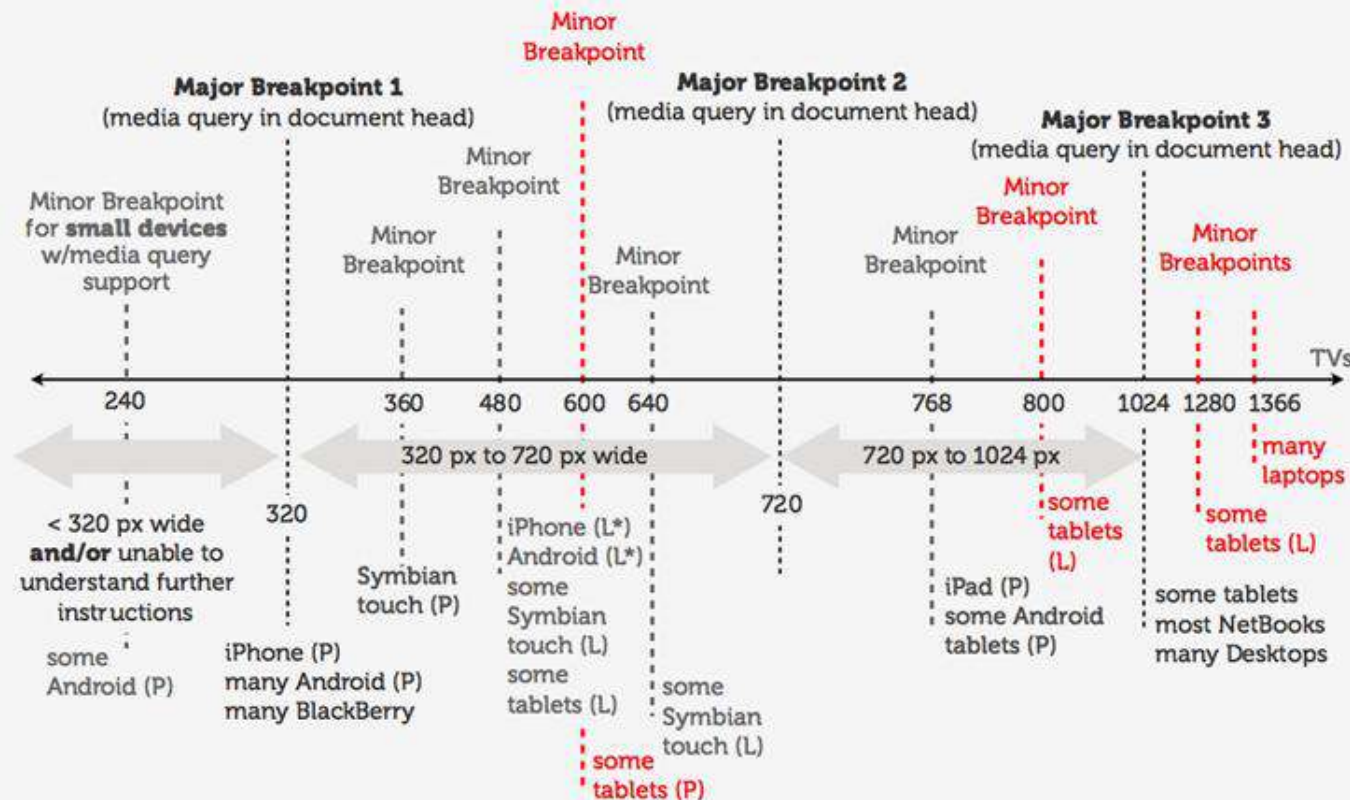
Пусть вы используете смартфон с определенными настройками экрана. При открытии неадаптивной страницы содержимое начинает смещаться или уползать. Брейкпоинты помогут этого избежать: отображения страницы и ее элементов будет происходить в зависимости от ширины экрана посетителя сайта. Элементы могут увеличиваться, уменьшаться, вовсе исчезать или добавляться на страницу.



Размеры макетов. Брейкпоинты

«Каноничные» значения для адаптивного дизайна:

- смартфоны: от 320px / 480 / выше;
- экран монитора ПК: от 1280px и выше;
- планшеты: от 768px и выше;
- нетбуки: 1024px.

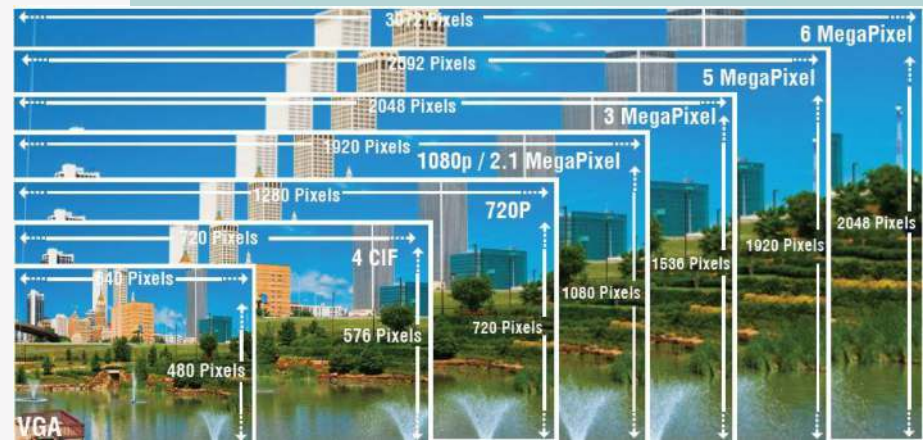
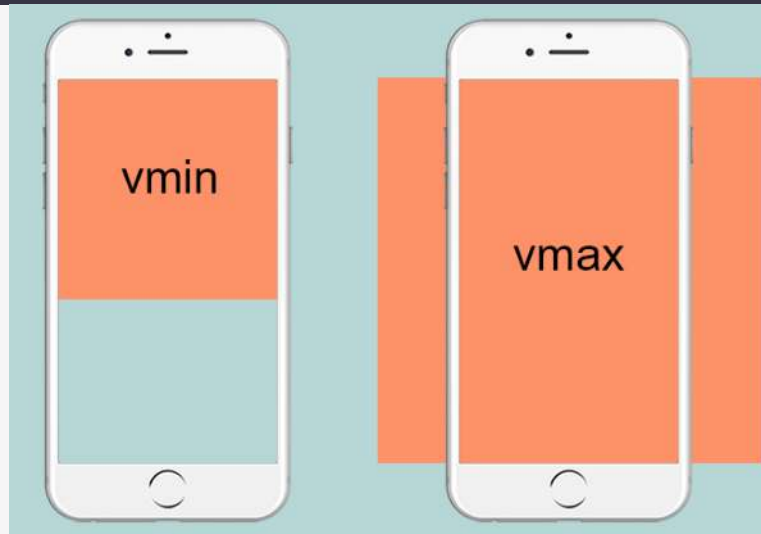


Относительность координат, размеров, масштабов

Разнообразие экранов и устройств не позволяет использовать точные единицы измерения. Говоря об адаптивном дизайне, нужно понимать, что в качестве таких единиц используются относительные значения. Потому что на каждом экране существует своя собственная плотность пикселей.

Так, разрешение 360 x 240 пикселей будет абсолютно по-разному выглядеть на экране планшета и телевизора.

Именно поэтому адаптивный дизайн использует относительные значения. Например: координаты/размеры/масштабы блока определяются по отношению к верхней границе или по отношению к любому другому элементу.



Вложенность

На любой странице присутствует много разных объектов. Относительное их позиционирование – это не только достоинство, но и недостаток, так как возникают сложности с расположением интерактивных объектов касательно друг друга.

Обычно эту проблему решают при помощи

блок-контейнера

– в него отправляют

всю группу

элементов

`<header>`

`<nav>`

`<header>`

`<article>`

`<header>`

`<section>`

`<section>`

`<footer>`

Wikitechy

`<aside>`

`<section>`

`<section>`

`<section>`

`<footer>`

Поточность

Имеется в виду отсутствие сдвигов блоков. При приеме сайта обратите внимание: если блоки смещаются, то просмотр страницы становится настоящим испытанием.

$> 640px$

$\leq 640px$
 $> 320px$

$\leq 320px$

Шрифты

Самые красивые шрифты, кроме достоинств, имеют некоторые недостатки. Да, встраиваемые шрифты – это красиво, и они эффективно отделяют ваш сайт от серой массы и десятков конкурентов, которые используют системные шрифты. Но у встраиваемых шрифтов есть существенный минус: кроме того, что многие из них платные, они также увеличивают время загрузки страницы для посетителя. Здесь придется искать компромисс между двумя типами шрифтов, либо делать идеальную оптимизацию всего сайта, чтобы добиться наилучшей скорости загрузки страниц.



Графические форматы

С графикой существует простое правило. Когда на изображении присутствует минимальная детализация – выбираем вектор. Когда требуется максимальная детализация – выбираем растр.

В любом случае, абсолютно все изображения на сайте должны использовать компрессию.



Отзывчивый дизайн — часть адаптивного?

Отзывчивый и адаптивный дизайн – это не два конкурирующих подхода, а разные способы проектирования интерфейсов (UX/UI), которые помогают подстроить их под экраны разного размера. Существует точка зрения, предложенная Вильями Салминеном в 2012 году:

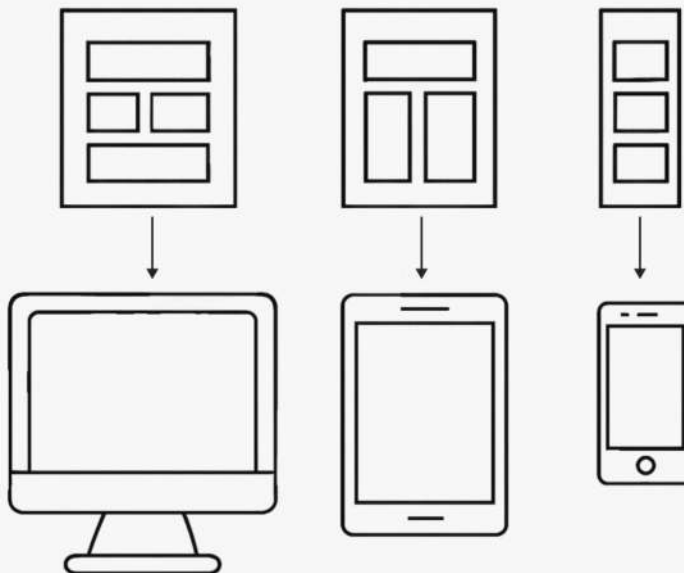
- **Адаптивный дизайн** – это широкая концепция, которая включает в себя не только макеты, но и учёт условий использования: тип устройства, соединение, доступность и прочее.
- **Отзывчивый дизайн** в этой системе – один из инструментов адаптации, отвечающий за перестройку интерфейса по ширине экрана.

Во многих источниках эти термины рассматриваются как отдельные, но близкие по цели подходы. Разница между ними в том, что один опирается на гибкий универсальный макет с медиазапросами, другой – на набор фиксированных шаблонов для разных экранов.

Адаптивный дизайн – это подход, при котором для каждого диапазона размеров экрана создается отдельный макет. Пользователю показывается не «растянутый» универсальный шаблон, а заранее подготовленная оптимизированная версия.

Схема адаптивного дизайна

для каждого устройства
используется отдельный макет,
заранее оптимизированный под
ширину экрана и тип устройства.



Адаптивный дизайн часто
следует принципу progressive
enhancement – «постепенного
улучшения».

Адаптивный дизайн (2.0)

Отзывчивый или адаптивный?

Отзывчивый дизайн – метод, в котором страница автоматически перестраивается под размер экрана пользователя. И слово «перестраивается» как раз и отражает главную разницу между адаптивным и отзывчивым дизайном.

Схема ОТЗЫВЧИВОГО дизайна

один гибкий макет
подстраивается

под разные устройства – от
настольных экранов
до планшетов и смартфонов



В отзывчивом дизайне используются три приёма:

Гибкая сетка (fluid grid) – элементы размещаются не в пикселях, а в процентах. Это позволяет блокам автоматически сжиматься или растягиваться вместе с шириной экрана.

Медиа-запросы (media queries) – это правила, которые применяют разные стили оформления в зависимости от ширины экрана.

Адаптивные элементы – картинки, кнопки и блоки, которые автоматически подстраиваются под экран.

Отзывчивый дизайн

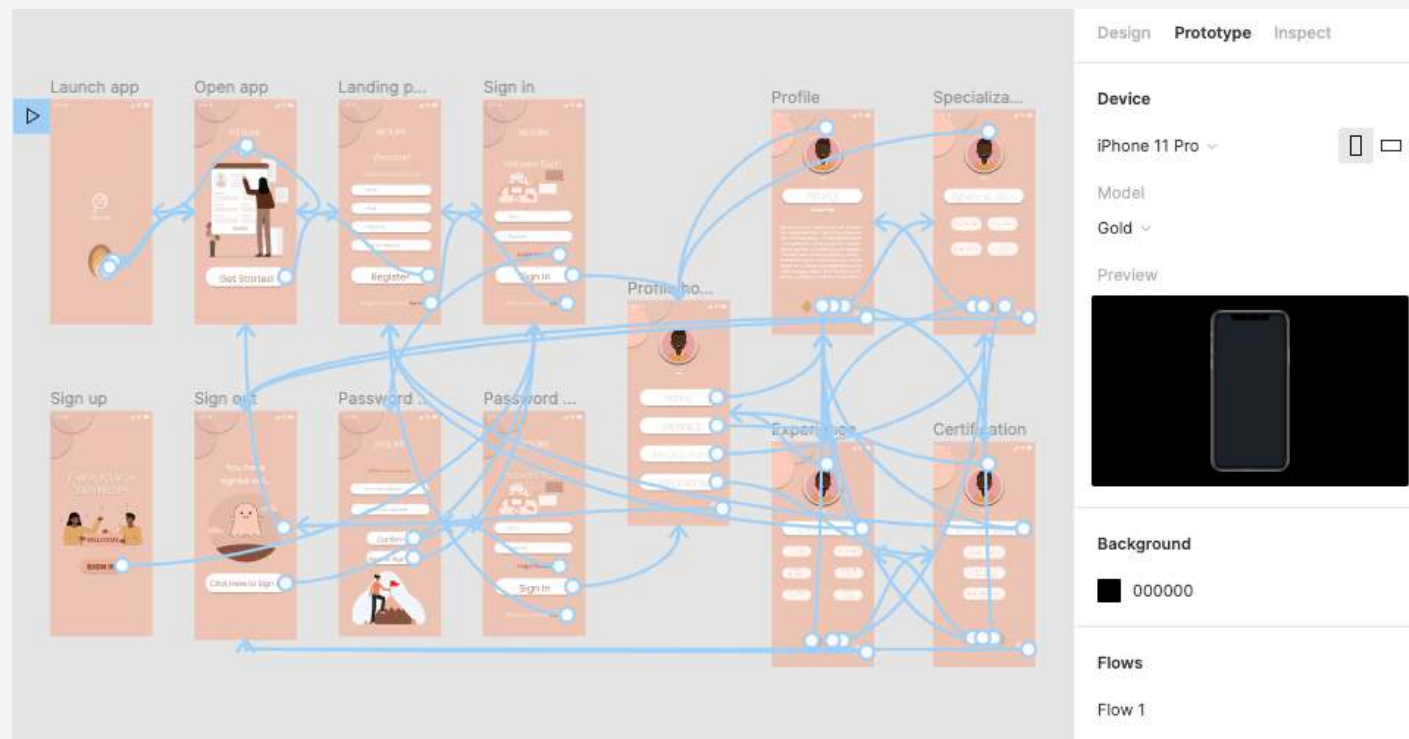
По сути, это гибрид фиксированной и жидкой вёрстки, при котором макет реагирует на ширину экрана, сохраняя функциональность и структуру.

Отзывчивый или адаптивный?

Технические требования

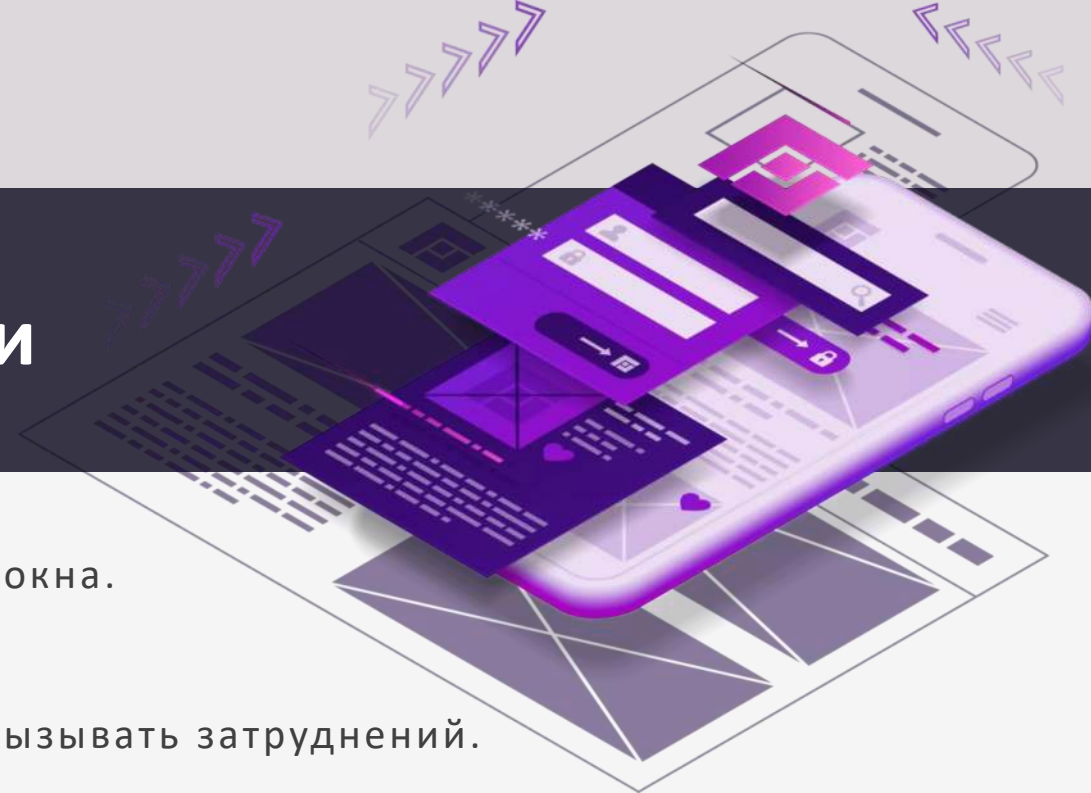
Разработка мобильной версии ставит своей целью решить все проблемы, возникающие у мобильной аудитории сайта:

- создать условия для удобного просмотра страниц;
- сделать комфортной навигацию и взаимодействие с элементами сайта;
- увеличить скорость загрузки страниц для мобильных устройств.



Чек-лист мобильной версии

1. Нежелательны поп-апы и другие всплывающие окна.
2. Формы должны быть максимально короткими.
3. Скроллинг и увеличение страницы не должны вызывать затруднений.
4. Номер телефона должен быть активным элементом страницы.
5. Используйте автозаполнение – оно удобно для тех, кто делает заказ повторно.
6. Меню должно быть доступно с любой страницы сайта.
7. Кнопка домой /главная должна быть доступна с любой страницы сайта.
8. Шрифт не должен быть очень мелким. Минимальный размер символов – 12px.
9. Расстояние между интерактивными элементами должно составлять минимум 28px.
10. Кнопки должны быть достаточно крупными.
Не делайте их слишком маленькими, по ним будет сложно попасть.

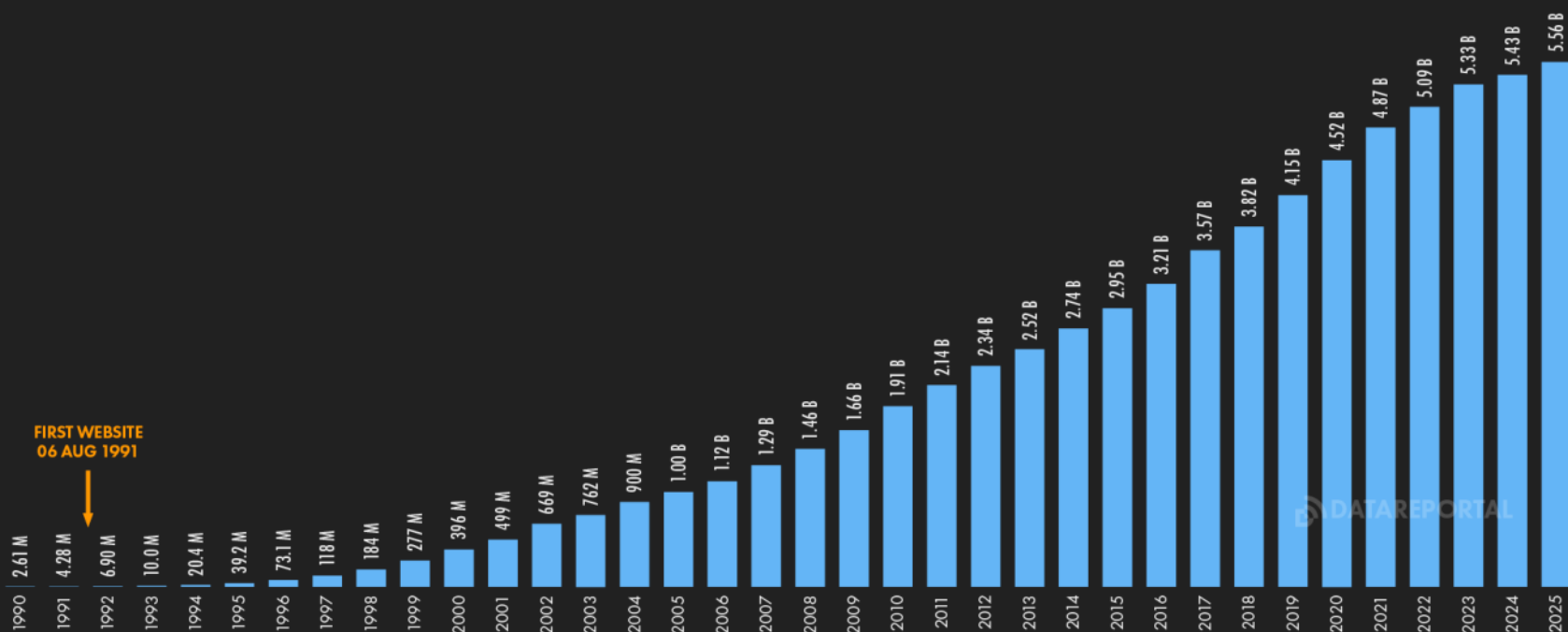


Для кого нужны сайты?

FEB
2025

INTERNET USE TIMELINE

NUMBER OF INDIVIDUALS USING THE INTERNET OVER TIME



FIRST WEBSITE
06 AUG 1991

SOURCES: KEPIOS ANALYSIS; ITU; GSMA INTELLIGENCE; EUROSTAT; GOOGLE'S ADVERTISING RESOURCES; CNNIC; KANTAR & IAMA; GOVERNMENT RESOURCES; UNITED NATIONS. **COMPARABILITY:** SOURCE AND BASE CHANGES. ALL FIGURES USE THE LATEST AVAILABLE DATA, BUT SOME SOURCES DO NOT PUBLISH REGULAR UPDATES, SO FIGURES FOR RECENT PERIODS MAY UNDER-REPRESENT ACTUAL USE. SEE [NOTES ON DATA](#).

we
are
social

Meltwater

В современном веб-дизайне сайт должен корректно работать на всех устройствах – от смартфонов до больших экранов и наоборот.

Число людей, использующих веб-ресурсы растет ежегодно.

<https://datareportal.com/reports/digital-2025-global-overview-report>

Мобильный трафик

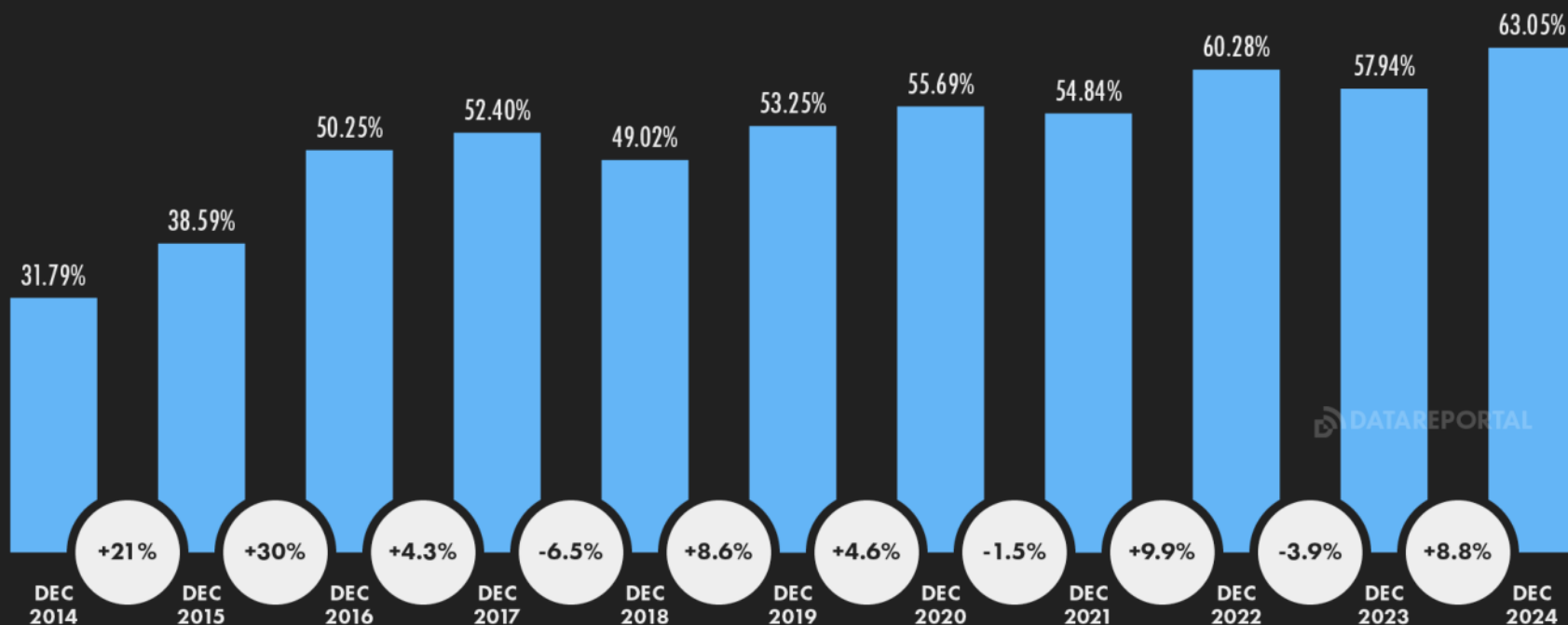
FEB
2025

MOBILE'S SHARE OF WEB TRAFFIC (YOY)

SHARE OF TOTAL WEB TRAFFIC (PERCENTAGE OF WEB PAGE REQUESTS) ORIGINATING FROM WEB BROWSERS RUNNING ON MOBILE PHONES



GLOBAL OVERVIEW



SOURCE: STATCOUNTER. **NOTES:** FIGURES REPRESENT THE NUMBER OF WEB PAGES SERVED TO WEB BROWSERS RUNNING ON MOBILE PHONES COMPARED WITH THE TOTAL NUMBER OF WEB PAGES SERVED TO WEB BROWSERS RUNNING ON ANY DEVICE. PERCENTAGE CHANGE VALUES IN THE WHITE CIRCLES REPRESENT RELATIVE CHANGE (I.E. AN INCREASE OF 20% FROM A STARTING VALUE OF 50% WOULD EQUAL 60%, NOT 70%).

we
are
social

Meltwater

Переход на мобильные устройства также очевиден. Согласно анализу Statcounter, в декабре 2024 года на мобильные устройства приходилось более 63% запросов к веб-страницам.

Эта почти в два раза превышает, который мы наблюдали 10 лет назад. И хотя доля устройств меняется, по-прежнему наблюдается явная тенденция в пользу мобильных устройств.

<https://datareportal.com/reports/digital-2025-global-overview-report>

FEB
2025

MAIN REASONS FOR USING THE INTERNET

PRIMARY REASONS WHY INTERNET USERS IN EACH AGE GROUP USE THE INTERNET



AGE 16 TO 24	AGE 25 TO 34	AGE 35 TO 44	AGE 45 TO 54	AGE 55 TO 64	AGE 65+*
CONTACT FRIENDS & FAMILY 62.1%	CONTACT FRIENDS & FAMILY 58.7%	FIND INFORMATION 62.1%	FIND INFORMATION 65.2%	FIND INFORMATION 68.1%	FIND INFORMATION 81.7%
FIND INFORMATION 61.1%	FIND INFORMATION 58.5%	CONTACT FRIENDS & FAMILY 59.7%	CONTACT FRIENDS & FAMILY 59.3%	FOLLOW NEWS & EVENTS 61.2%	FOLLOW NEWS & EVENTS 66.9%
WATCH VIDEOS & SHOWS 59.3%	WATCH VIDEOS & SHOWS 56.2%	FOLLOW NEWS & EVENTS 56.3%	FOLLOW NEWS & EVENTS 58.9%	CONTACT FRIENDS & FAMILY 61.1%	CONTACT FRIENDS & FAMILY 62.7%
LISTEN TO MUSIC 55.3%	FOLLOW NEWS & EVENTS 51.5%	WATCH VIDEOS & SHOWS 55.1%	WATCH VIDEOS & SHOWS 53.6%	LEARN HOW TO DO THINGS 53.2%	LEARN HOW TO DO THINGS 60.9%
EDUCATION & STUDY 52.4%	LEARN HOW TO DO THINGS 48.6%	LEARN HOW TO DO THINGS 50.0%	LEARN HOW TO DO THINGS 51.8%	WATCH VIDEOS & SHOWS 49.3%	RESEARCH BRANDS 57.0%
LEARN HOW TO DO THINGS 51.7%	FIND NEW IDEAS 48.4%	FIND NEW IDEAS 48.0%	RESEARCH BRANDS 47.4%	RESEARCH BRANDS 47.5%	RESEARCH PLACES & TRAVEL 50.7%
FIND NEW IDEAS 50.5%	LISTEN TO MUSIC 47.8%	RESEARCH BRANDS 46.1%	FIND NEW IDEAS 45.7%	FILL SPARE TIME & BROWSING 44.8%	RESEARCH HEALTH 46.9%
FOLLOW NEWS & EVENTS 49.5%	FILL SPARE TIME & BROWSING 44.1%	LISTEN TO MUSIC 45.4%	FILL SPARE TIME & BROWSING 44.4%	RESEARCH PLACES & TRAVEL 41.7%	MANAGE FINANCES 42.3%
FILL SPARE TIME & BROWSING 49.3%	RESEARCH BRANDS 43.4%	FILL SPARE TIME & BROWSING 43.6%	LISTEN TO MUSIC 43.1%	FIND NEW IDEAS 41.5%	FILL SPARE TIME & BROWSING 40.9%
GAMING 42.8%	EDUCATION & STUDY 39.5%	RESEARCH PLACES & TRAVEL 39.5%	RESEARCH PLACES & TRAVEL 40.5%	RESEARCH HEALTH 40.5%	WATCH VIDEOS & SHOWS 35.6%

SOURCE: GWI (Q3 2024). *NOTE: DATA FOR AUDIENCES AGED 65+ ARE NOT YET AVAILABLE IN ALL COUNTRIES, SO FINDINGS FOR AUDIENCES AGED 65+ MAY NOT BE DIRECTLY COMPARABLE WITH THOSE FOR OTHER AGE GROUPS. COMPARABILITY: CHANGES IN AUDIENCE COMPOSITION AND SURVEY METHODOLOGY. SEE [NOTES ON DATA](#).

Адаптивный VS Отзывчивый

	Адаптивный	Отзывчивый
Подход к разработке	Несколько фиксированных макетов для разных устройств	Один гибкий макет, подстраивающийся под любые размеры экрана
Влияние на производительность	Более производительный, так как загружается только нужная версия макета	Требует больше ресурсов для гибкой адаптации всех элементов
Удобство для пользователя	Оптимально для конкретного устройства, но могут возникнуть скачки при смене разрешений	Плавное изменение макета, удобное взаимодействие на всех экранах
Гибкость макета	Фиксированные макеты для каждого разрешения экрана	Макет гибкий и изменяется динамически
SEO-оптимизация	Одноразовая настройка SEO для всех версий макета, не требует дополнительных настроек и избегает дублирование контента	Единая вервия сайта улучшает SEO, избегает дублирование контента
Сложность разработки	Требует разработки и тестирования нескольких макетов	Разовая разработка, но сложнее отладка
Обновление контента	Необходимо обновлять контент на каждой версии макета	Единое обновление для всех устройств
Тестирование	Необходимо тестировать несколько макетов на разных устройствах	Проще тестировать один макет, который адаптируется к любому экрану
Затраты на разработку	Дороже из-за необходимости создания нескольких макетов	Менее затратный, так как создается один макет для всех экранов

Адаптивный VS Мобильный

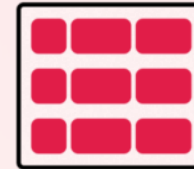
	Адаптивный	Мобильный
Подход к разработке	Несколько фиксированных макетов для разных устройств	Создание отдельной версии сайта для мобильных устройств
База кода	Единая база кода для всех версий сайта	Отдельные базы кода для мобильной и десктопной версии
Управление контентом	Упрощение управления, так как контент синхронизирован	Необходимо обновлять контент на каждой версии отдельно
SEO-оптимизация	Единая версия сайта улучшает SEO	Проблема дублирования контента негативно влияет на SEO
Производительность	Эффективный, так как загружается один макет	Загружается быстрее за счет упрощенного интерфейса
Затраты на разработку и поддержку	Меньше затрат на разработку и поддержку	Требует больших затрат на разработку и поддержку двух версий сайта
Оптимизация под устройства	Автоматическая адаптация под различные устройства	Оптимизация под конкретные мобильные устройства
URL	Один URL для всех устройств	Отдельный URL для мобильной версии

2

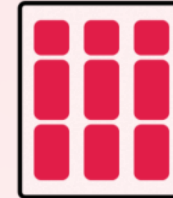
FLEXBOX

CSS Flexbox

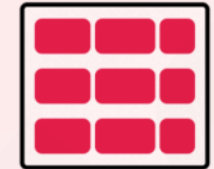
flex-direction



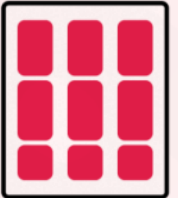
row



column



row-reverse



column-reverse

align-items



flex-start



center



flex-end



stretch

justify-content



flex-start



center



flex-end



space-between



space-around



space-evenly

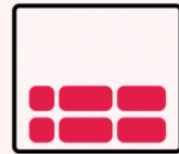
align-content



flex-start



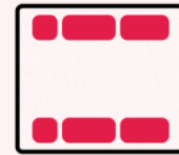
center



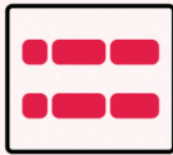
flex-end



stretch



space-between



space-around

Flexbox

Гибкая компоновка CSS, известная как **Flexbox**, представляет собой модель веб-макета CSS3. Она позволяет автоматически упорядочивать адаптивные элементы внутри контейнера

в зависимости от размера экрана устройства. Долгое время единственными надёжными инструментами CSS вёрстки были такие способы как Float (обтекание) и позиционирование. С их помощью сложно или невозможно достичь следующих простых требований к макету:

- Вертикального выравнивания блока внутри родителя.
- Оформления всех детей контейнера так, чтобы они распределили между собой доступную ширину/высоту, независимо от того, сколько ширины/высоты доступно.
- Сделать все колонки в макете одинаковой высоты, даже если наполнение в них различно



Flexbox даёт один из наиболее эффективных способов расстановки и распределения места между элементами внутри контейнера, даже если их размер неизвестен или динамичен (собственно, по этому его и называют flex, от слова flexible, что по-англ.— гибкий и уступчивый).

Цель: предоставление возможности изменения своих элементов по ширине и высоте, для того, чтобы они максимально эффективно умещались в доступном месте родительского контейнера, в частности — это удобно в тех случаях, когда нужно соответствовать всем типам дисплеев устройств и размерам экранов.

Flex контейнер расширяет вложенные элементы для того, чтобы заполнить доступное пространство или же урезает их, чтобы избежать переполнения.

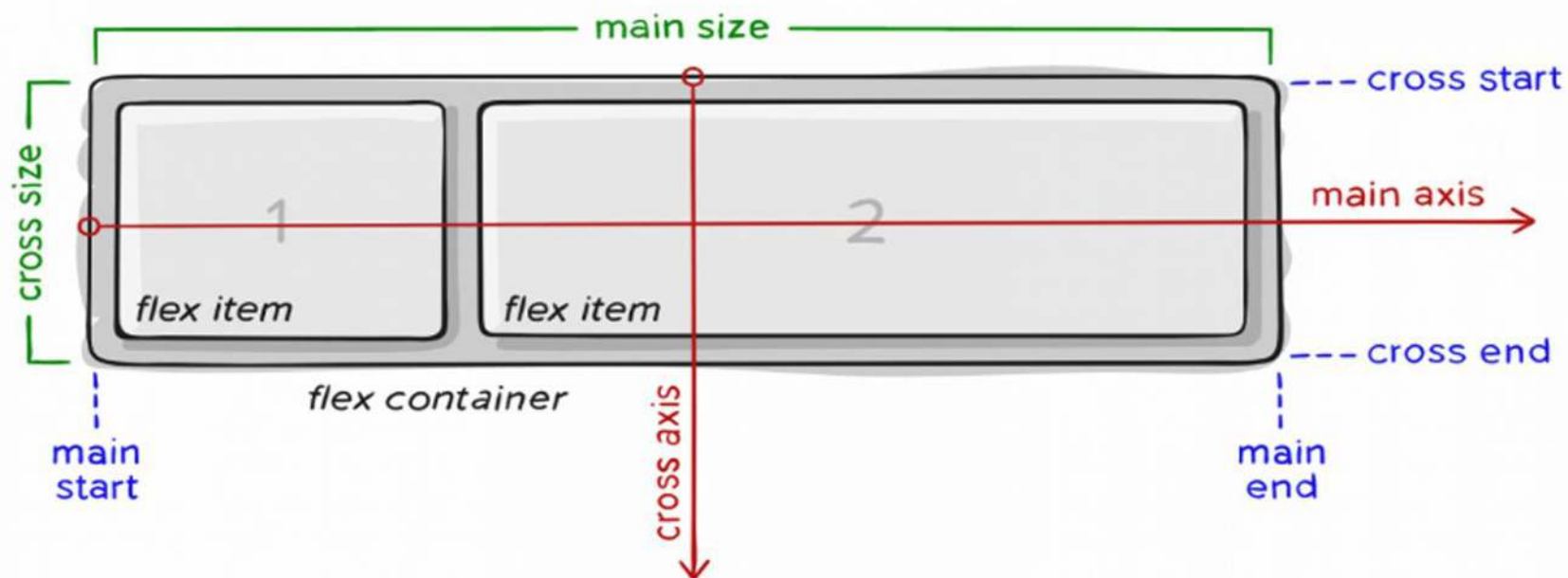
Основная цель Flexbox

Flexbox



Flexbox

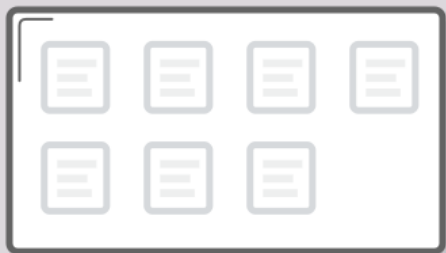
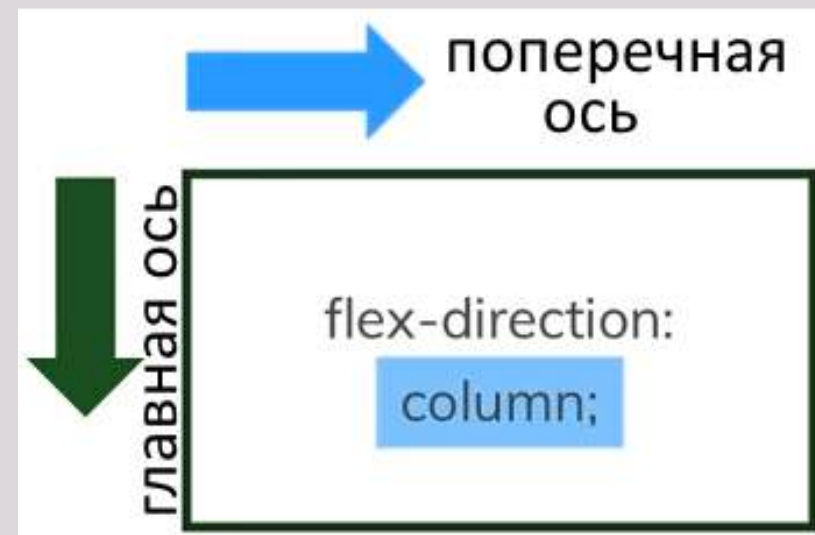
Поскольку flexbox — это целый модуль, а не одно свойство, он включает в себя множество элементов с набором свойств. Некоторые из них предназначены для установки в контейнере (родительский элемент принято называть «flex контейнер»), в то время как другие предназначены для установки в дочерних элементах (так называемые «flex элементы»).



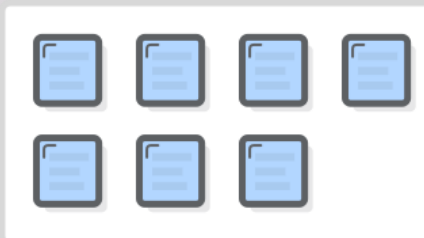
Основные термины

Флекс-контейнер: элемент, к которому применяется свойство `display: flex`.

Вложенные в него элементы подчиняются правилам раскладки флексов.



“FLEX CONTAINER”



“FLEX ITEMS”

Флекс-элемент: элемент, вложенный во флекс-контейнер.

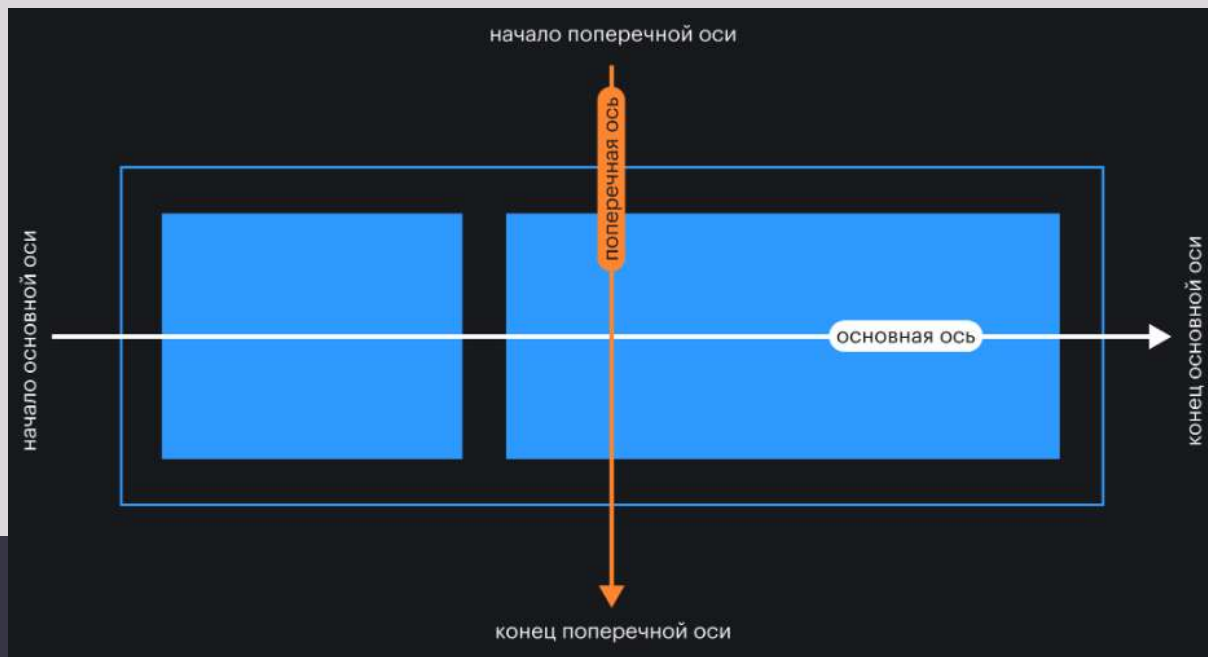
Основная ось: основная направляющая флекс-контейнера, вдоль которой располагаются флекс-элементы.

Поперечная (перпендикулярная, побочная) ось: ось, идущая перпендикулярно основной

Основные термины

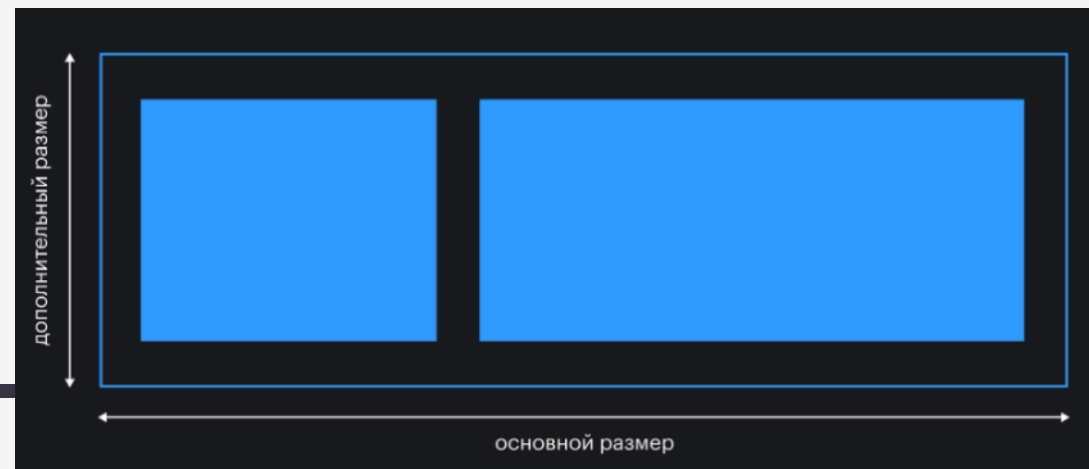
Начало / конец основной оси: точки в начале и в конце основной оси соответственно. Это пригодится нам для выравнивания флекс-элементов.

Начало / конец поперечной оси: точки в начале и в конце поперечной оси соответственно.



Размер по основной оси (основной размер): размер флекс-элемента вдоль основной оси. Это может быть ширина или высота в зависимости от направления основной оси.

Размер по поперечной оси (поперечный размер): размер флекс-элемента вдоль поперечной оси. Это может быть ширина или высота в зависимости от направления поперечной оси. Этот размер всегда перпендикулярен основному. Если основной размер – ширина, то поперечный – высота, и наоборот.



Свойства флекс-контейнера

display

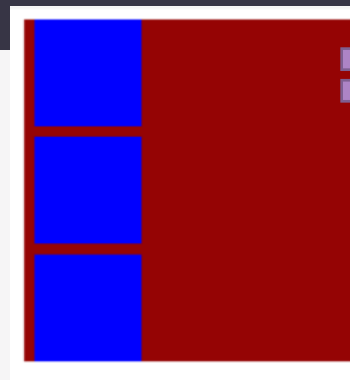
Когда мы задаём элементу значение **flex** для свойства **display**, мы превращаем этот элемент в флекс-контейнер. Внутри него начинает действовать флекс-контекст, его дочерние элементы начинают подчиняться свойствам флексбокса.

Снаружи флекс-контейнер выглядит как блочный элемент – занимает всю ширину родителя, следующие за ним элементы в разметке переносятся на новую строку.

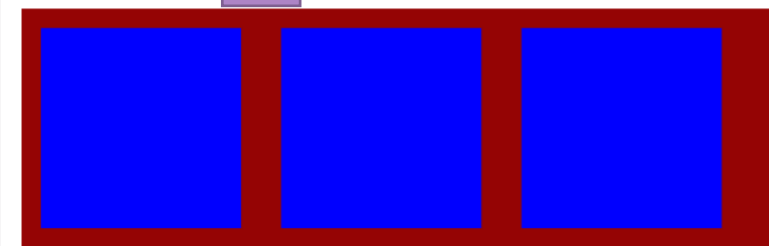
```
<body>
  <div class="red">
    <div class="blue"></div>
    <div class="blue"></div>
    <div class="blue"></div>
  </div>
</body>
</html>
```

```
.blue {
  width: 100px;
  height: 100px;
  background-color: blue;
  margin: 10px;
}

.red {
  background-color: rgb(149, 4, 4);
  margin: 10px;
}
```



```
.red {
  background-color: rgb(149, 4, 4);
  margin: 10px;
  display: flex;
}
```



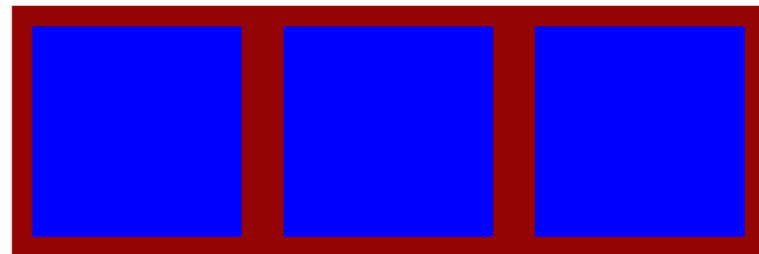
display

Если контейнеру задано значение **inline-flex**, то снаружи он начинает вести себя как строчный (инлайн) элемент — размеры зависят только от внутреннего контента, встаёт в строку с другими элементами.

Внутри это ровно такой же флекс-контейнер, как и при предыдущем значении.

```
<body>
  <div class="red">
    <div class="blue"></div>
    <div class="blue"></div>
    <div class="blue"></div>
  </div>
</body>
</html>
```

```
.red {
  background-color: ■ rgb(149, 4, 4);
  margin: 10px;
  display: inline-flex;
}
```



flex-direction

- это свойство управления направлением основной и поперечной осей.

Возможные значения:

row (по умолчанию) — основная ось идёт горизонтально слева направо, поперечная ось идёт вертикально сверху вниз.



```
.parent {  
  display: flex;  
  flex-direction: row;  
  height: 100%;  
}
```

row-reverse — основная ось идёт горизонтально справа налево, поперечная ось идёт вертикально сверху вниз.



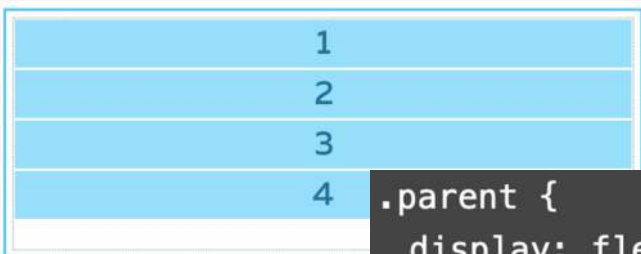
```
.parent {  
  display: flex;  
  flex-direction: row-reverse;  
  height: 100%;  
}
```

flex-direction

- это свойство управления направлением основной и поперечной осей.

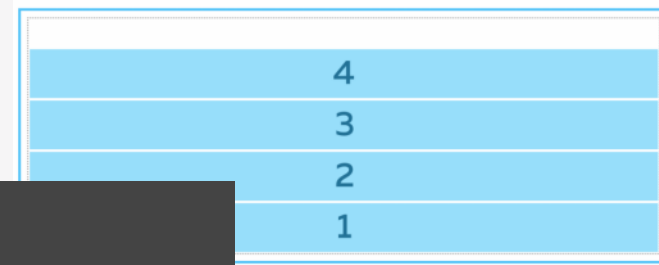
Возможные значения:

column — основная ось идёт вертикально сверху вниз, поперечная ось идёт горизонтально слева направо.



```
.parent {  
  display: flex;  
  flex-direction: column;  
  height: 100%;  
}
```

column-reverse — основная ось идёт вертикально снизу вверх, поперечная ось идёт горизонтально слева направо.



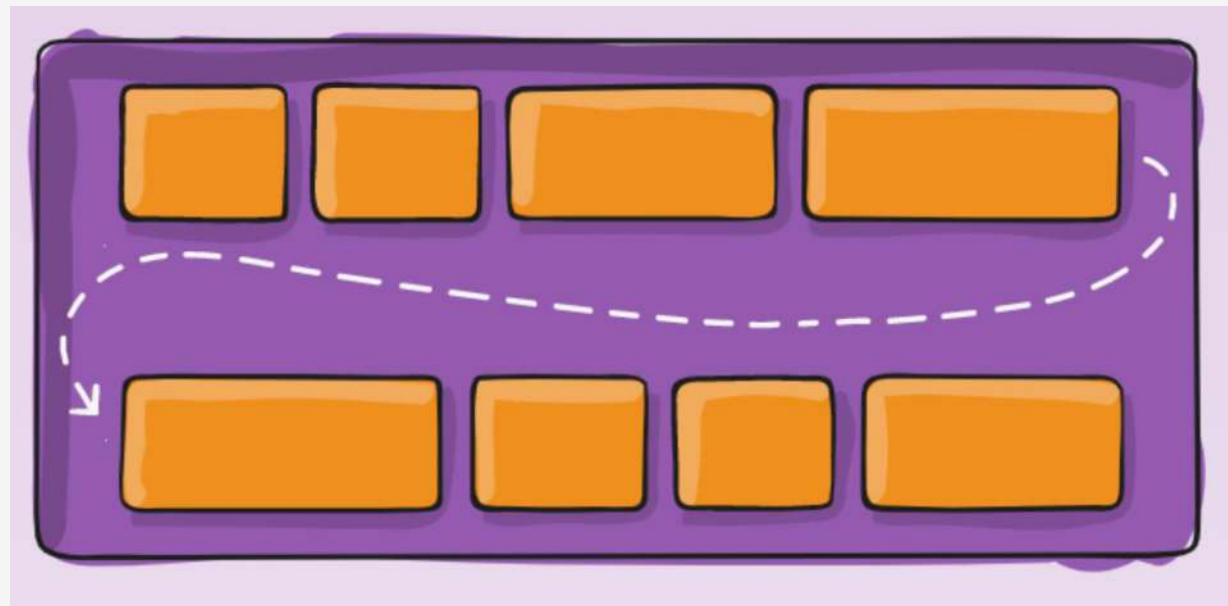
```
.parent {  
  display: flex;  
  flex-direction: column-reverse;  
  height: 100%;  
}
```

flex-wrap = перенос строк внутри контейнера

Свойство flex-wrap Указывает, следует ли флексам располагаться в одну строку или можно занять несколько строк. Если перенос строк допускается, то свойство также позволяет контролировать направление, в котором выкладываются строки.

Применяется к: flex контейнерам.

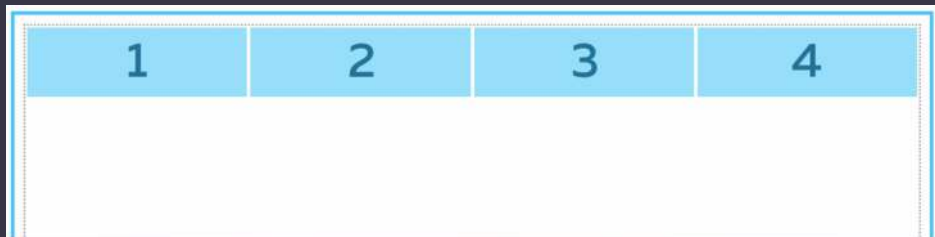
Значение по умолчанию: nowrap



flex-wrap

nowrap

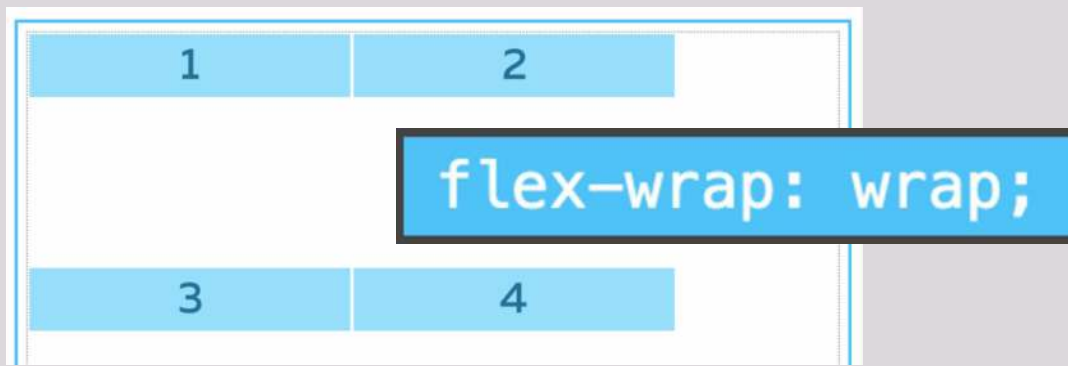
Флексы выстраиваются в одну линию.



```
.parent {  
  display: flex;  
  align-items: flex-start;  
  flex-wrap: nowrap;  
  height: 100%;  
}  
.child {  
  width: 40%;  
}
```

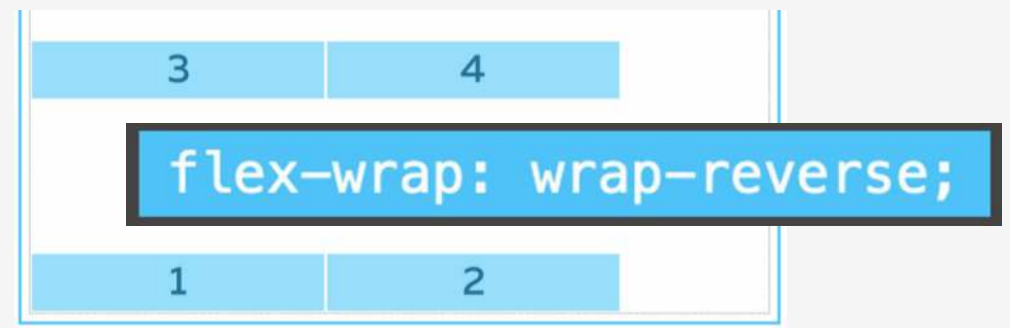
wrap

Флексы выстраиваются в несколько строк, их направление задаётся свойством **flex-direction**.



wrap-reverse

Флексы выстраиваются в несколько строк, в направлении, противоположном **flex-direction**.



flex-flow

Это свойство-шорткат
для одновременного
определения значений свойств
flex-direction и **flex-wrap**

```
.container {  
  width: 100%;  
  height: 320px;  
  display: flex;  
  flex-flow: column wrap;  
}
```

Start	Item 7
Item 1	Item 8
Item 2	Item 9
Item 3	— ↕ +
Item 4	End
Item 5	
Item 6	

```
.container {  
  width: 100%;  
  height: 320px;  
  display: flex;  
  flex-flow: row wrap;  
}
```

Start	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6	Item 7	Item 8
Item 9	Item 10	End						

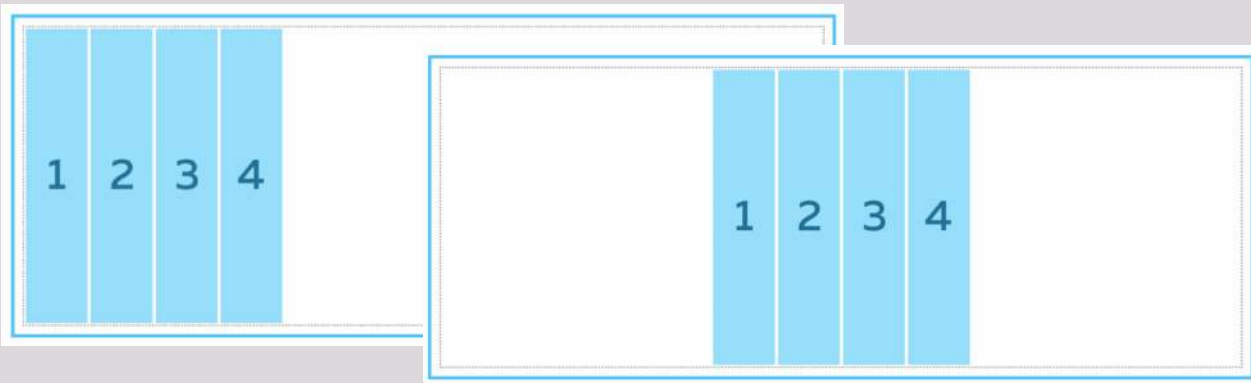
```
.container {  
  width: 100%;  
  height: 320px;  
  display: flex;  
  flex-flow: column-reverse wrap-reverse;  
}
```

	Item 6
	Item 5
End	Item 4
Item 10	Item 3
Item 9	Item 2
Item 8	Item 1
Item 7	Start

Justify-content

Свойство **justify-content** определяет, как браузер распределяет пространство вокруг флекс элементов вдоль главной оси контейнера.

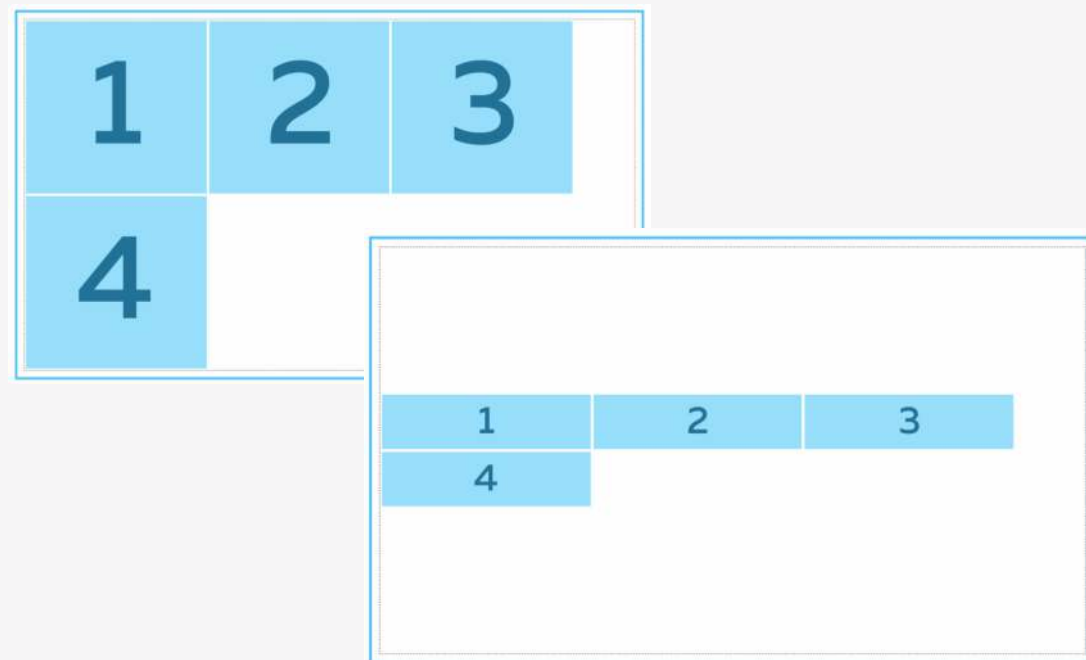
Это делается после того, как применяются размеры и автоматические отступы, за исключением ситуации, когда по крайней мере у одного элемента **flex-grow** больше нуля. При этом не остаётся свободного пространства для манипулирования.



Применяется к: flex контейнерам.
Значение по умолчанию: flex-start.

Align-content

Свойство **align-content** задаёт тип выравнивания строк внутри **flex** контейнера по поперечной оси при наличии свободного пространства.

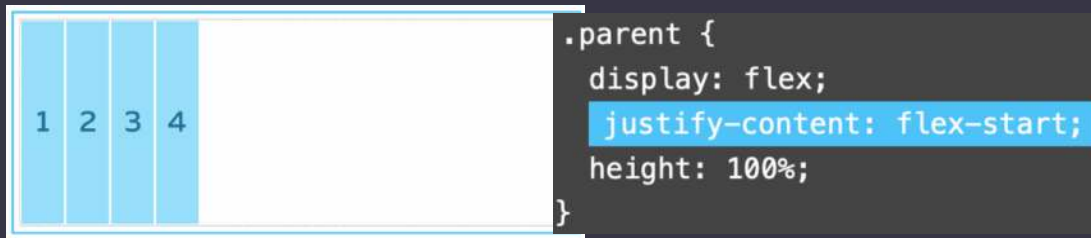


Применяется к: flex контейнерам.
Значение по умолчанию: stretch.

Justify-content

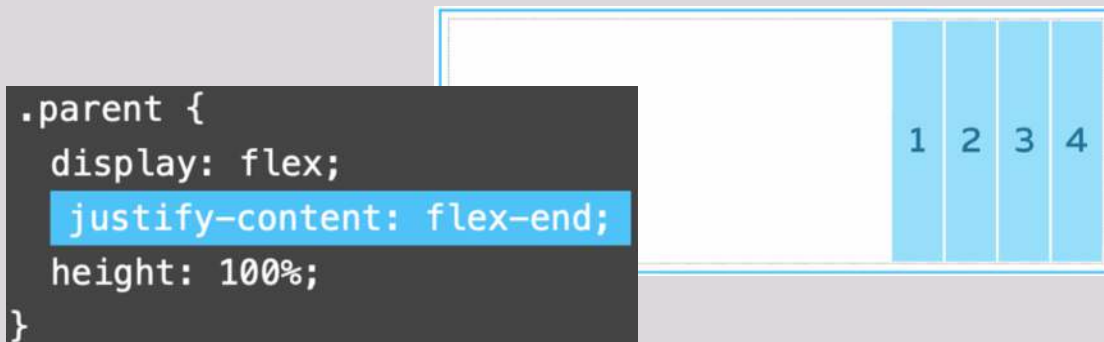
flex-start

Флексы прижаты к началу строки.



flex-end

Флексы прижаты к концу строки.



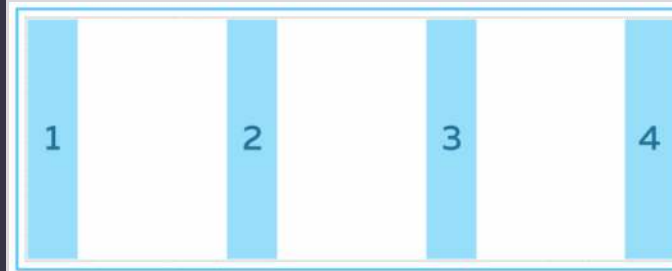
center

Флексы выравниваются по центру строки.



Justify-content

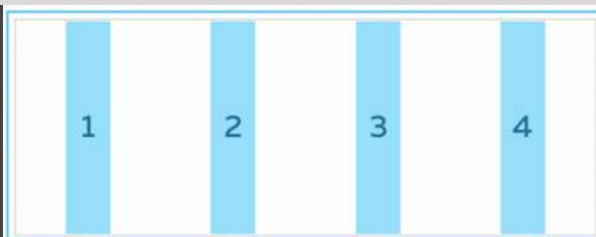
space-between. Флексы равномерно распределяются по всей строке. Первый и последний элемент прижимаются к соответствующим краям контейнера.



```
.parent {  
  display: flex;  
  justify-content: space-between;  
  height: 100%;  
}
```

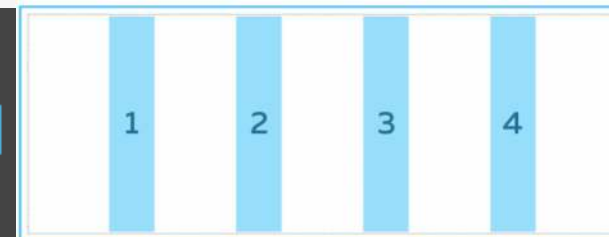
space-around. Флексы равномерно распределяются по всей строке. Пустое пространство перед первым и после последнего элементов равно половине пространства между двумя соседними элементами.

```
.parent {  
  display: flex;  
  justify-content: space-around;  
  height: 100%;  
}
```



space-evenly. Флексы распределяются так, что расстояние между любыми двумя соседними элементами, а также перед первым и после последнего, было одинаковым.

```
.parent {  
  display: flex;  
  justify-content: space-evenly;  
  height: 100%;  
}
```



Align-items

flex-start

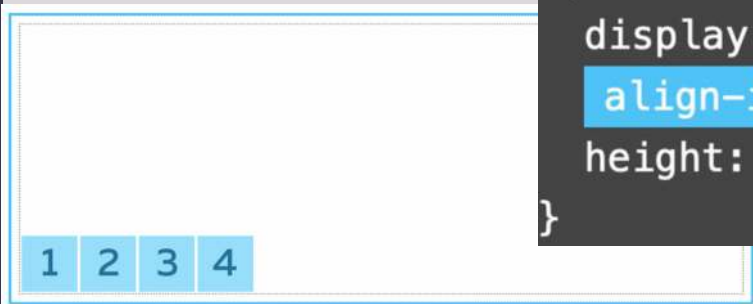
Флексы выравниваются в начале поперечной оси контейнера.



```
.parent {  
  display: flex;  
  align-items: flex-start;  
  height: 100%;  
}
```

flex-end

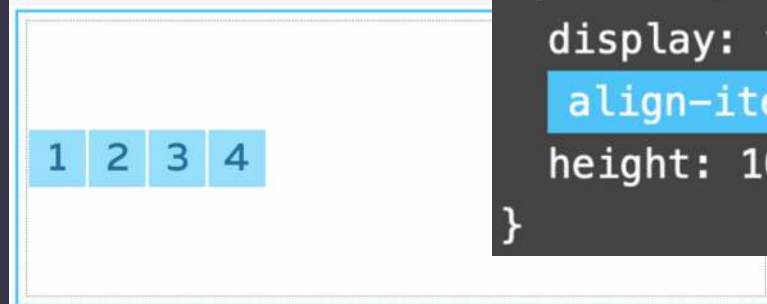
Флексы выравниваются в конце поперечной оси контейнера.



```
.parent {  
  display: flex;  
  align-items: flex-end;  
  height: 100%;  
}
```

center

Флексы выравниваются по линии поперечной оси.



```
.parent {  
  display: flex;  
  align-items: center;  
  height: 100%;  
}
```

Align-items

baseline

Флексы выравниваются по их базовой линии.



```
.parent {  
  display: flex;  
  align-items: baseline;  
  height: 100%;  
}
```

Stretch

Флексы растягиваются таким образом, чтобы занять всё доступное пространство контейнера



```
.parent {  
  display: flex;  
  align-items: stretch;  
  height: 100%;  
}
```

gap (краткая запись свойств row-gap и column-gap)

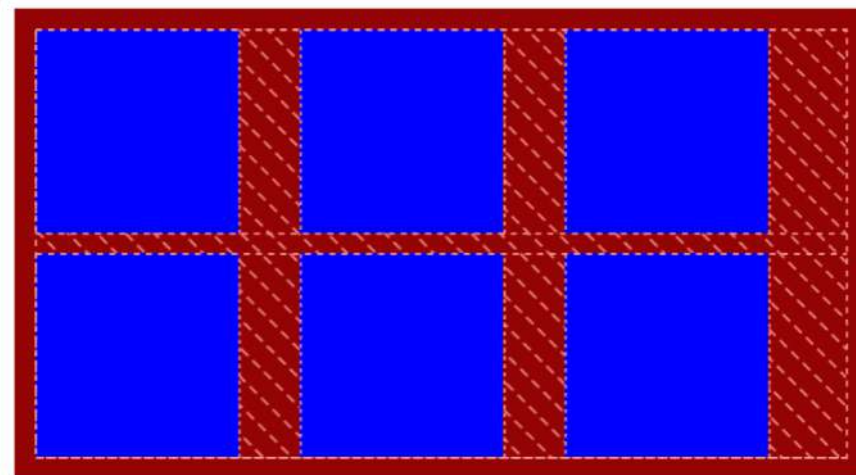
Может иметь одно или два значения. Если указано только одно, то **column-gap** автоматически равен **row-gap**. Если указаны два значения, то первое будет задавать отступы между рядами (**row-gap**), а второе — между колонками (**column-gap**).

Значения можно указывать в любых доступных единицах измерения, включая проценты.

Допускается использование функции **calc()**

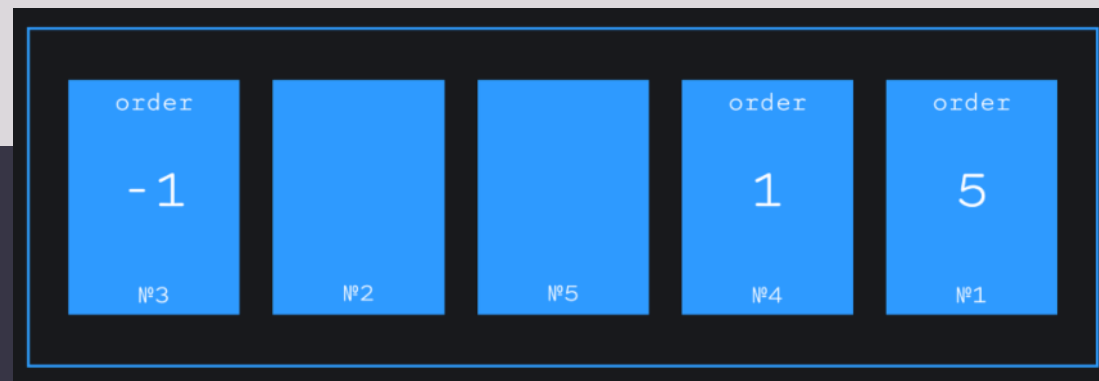
```
<body>
  <div class="red">
    <div class="blue"></div>
    <div class="blue"></div>
    <div class="blue"></div>
    <div class="blue"></div>
    <div class="blue"></div>
    <div class="blue"></div>
  </div>
</body>
</html>
```

```
.red {
  background-color: #990000;
  padding: 10px;
  display: flex;
  flex-flow: row wrap;
  flex-direction: row;
  flex-wrap: wrap;
  gap: 10px 30px;
  row-gap: 10px;
  column-gap: 30px;
}
```



Свойства флекс-элемента

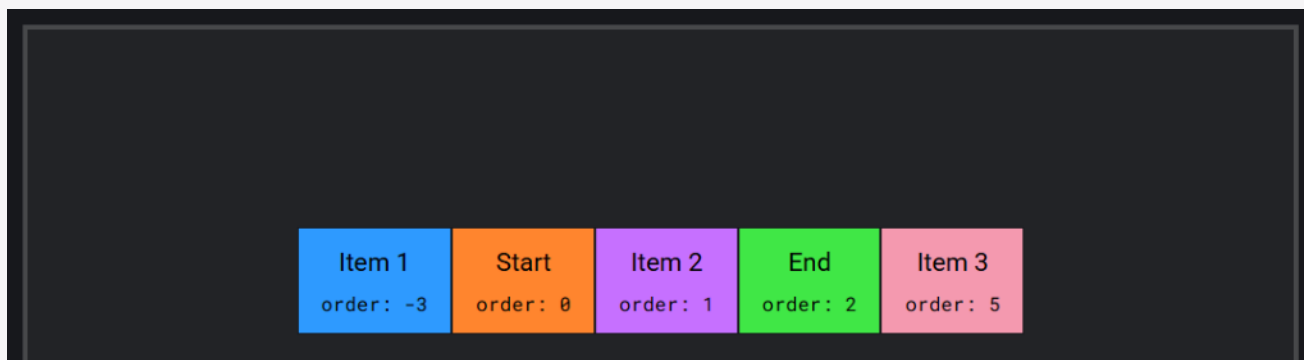
order



При помощи свойства **order** можно менять порядок отображения флекс-элементов внутри флекс-контейнера. По умолчанию элементы отображаются в том порядке, в котором они расположены в разметке, а значение свойства **order** равно 0.

Значение задаётся в виде целого отрицательного или положительного числа.

Элементы встают по возрастающей.



```
.container {  
  width: 100%;  
  height: 320px;  
  display: flex;  
  flex-direction: row;  
  flex-wrap: nowrap;  
  justify-content: center;  
  align-items: center;  
}
```

flex-grow

Указывает, может ли вырастать флекс-элемент при наличии свободного места, и насколько. По умолчанию значение = 0. Значением может быть любое положительное целое число (включая 0).

Если у всех флекс-элементов будет прописано **flex-grow: 1**, то свободное пространство в контейнере будет равномерно распределено между всеми.

Если при этом одному из элементов мы зададим **flex-grow: 2**, то он постарается занять в два раза больше свободного места, чем его соседи.

Эти свойства работают с пропорциональным разделением пространства, не с конкретными размерами

flex-shrink

flex-shrink противоположно flex-grow. Если в контейнере не хватает места для расположения всех элементов без изменения размеров, то flex-shrink указывает, в каких пропорциях элемент будет уменьшаться.

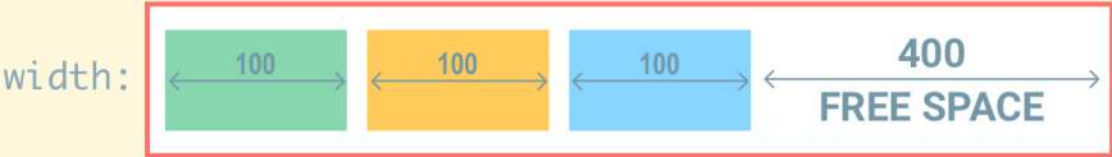
Чем больше значение у этого свойства, тем быстрее элемент будет сжиматься по сравнению с соседями, имеющими меньшее значение.

Значение по умолчанию = 1. Значением может быть любое целое ≥ 0 .

flex-grow

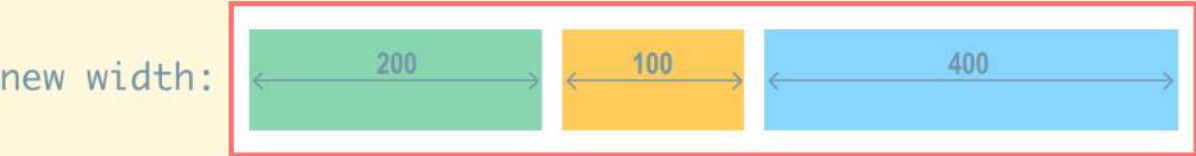
flex-grow calculation

$$\text{new width} = \left(\frac{\text{flex grow}}{\text{total flex grow}} \times \text{free space} \right) + \text{width}$$



calculation:

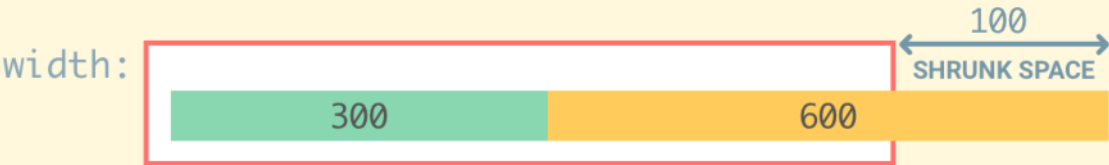
$$\left(\frac{1}{4} \times 400 \right) + 100 \quad \left(\frac{0}{4} \times 400 \right) + 100 \quad \left(\frac{3}{4} \times 400 \right) + 100$$



flex-shrink

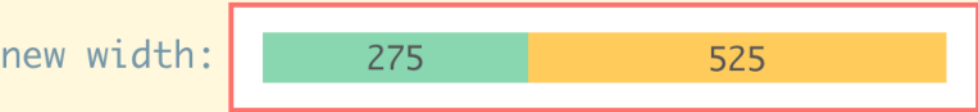
flex-shrink calculation

$$\text{new width} = \text{width} - (\text{shrunk space} \times \text{shrink ratio})$$



calculation:

$$\text{total shrink scaled width} = \sum (\text{width} \times \text{flex shrink})$$
$$(300 \times 4) + (600 \times 6) = 4800$$
$$\text{shrink ratio} = (\text{width} \times \text{flex shrink}) / \text{total shrink scaled width}$$
$$(300 \times 4) / 4800 = 0.25 \quad (600 \times 6) / 4800 = 0.75$$
$$\text{new width} = \text{width} - (\text{shrunk space} \times \text{shrink ratio})$$
$$300 - (100 \times 0.25) = 275 \quad 600 - (100 \times 0.75) = 525$$



flex-basis

flex-basis указывает на размер элемента до того, как свободное место будет распределено.

Значением может быть размер в любых относительных или абсолютных единицах, можно использовать ключевое слово **auto**. В этом случае при расчёте размера элемента будут приниматься во внимание значения свойств **width**, **max-width**, **min-width** или аналогичные свойства высоты, в зависимости от того, в каком направлении идёт основная ось.

Если никакие размеры не заданы, а свойству **flex-basis** установлено значение **auto**, то элемент занимает столько пространства, сколько нужно для отображения контента

flex-basis vs widths

```
.child {  
  ✓ flex-basis: 500px  
  width: 100px  
}
```

flex-basis wins

500px

```
.child {  
  ✓ min-width: 100px  
  flex-basis: 500px  
}
```

min-width wins

100px

```
.child {  
  ✓ max-width: 700px  
  flex-basis: 500px  
}
```

max-width wins

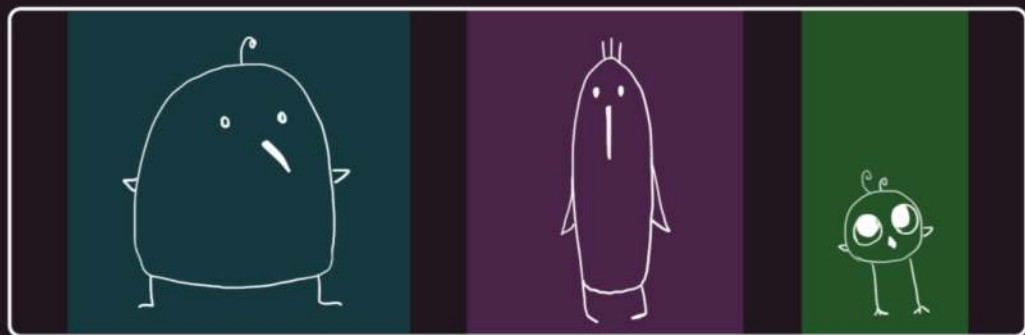
700px

flex

Свойство-шорткат, с помощью которого можно указать значение трёх свойств одновременно: **flex-grow**, **flex-shrink** и **flex-basis**. Первое значение является обязательным, остальные опциональны.

Значение по умолчанию: 0 1 auto, что расшифровывается как:

flex-grow: 0,
flex-shrink: 1,
flex-basis: auto



flex-basis: 200px

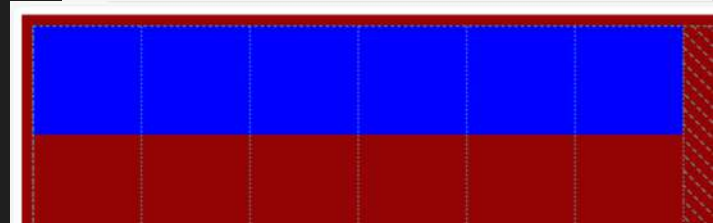
flex-basis: 150px

flex-basis: 100px

align-self

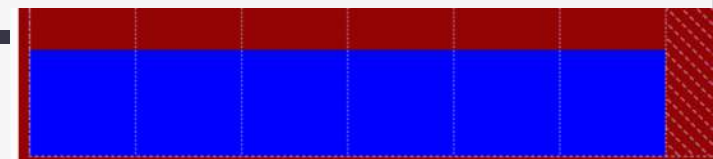
При помощи этого свойства можно выровнять один или несколько элементов иначе, чем задано у родителя. Например, в коде у родителя задано выравнивание вложенных элементов по верхнему краю родителя. А для дочерних задаём выравнивание по нижнему краю.

```
.red {  
  height: 400px;  
  background-color: red;  
  padding: 10px;  
  display: flex;  
  align-items: flex-start;  
}
```



```
.blue {  
  width: 100px;  
  height: 100px;  
  background-color: blue;  
}
```

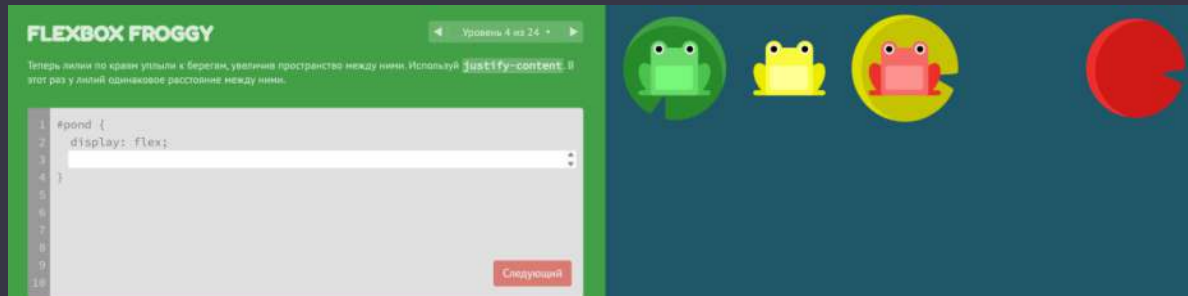
```
.blue {  
  width: 100px;  
  height: 100px;  
  background-color: blue;  
  align-self: flex-end;  
}
```



Освоить быстрее

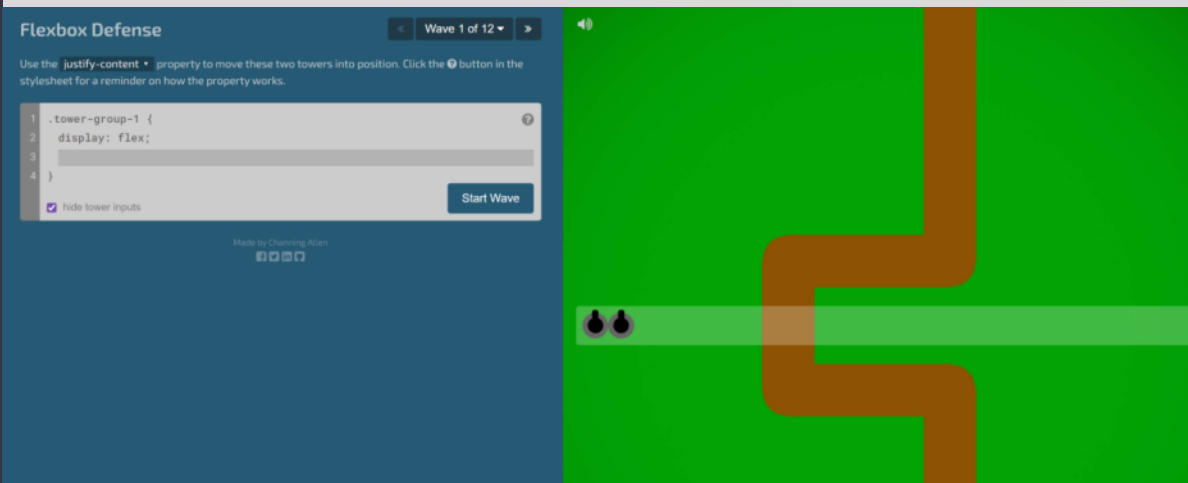
Flexbox Froggy

<https://flexboxfroggy.com/#ru>



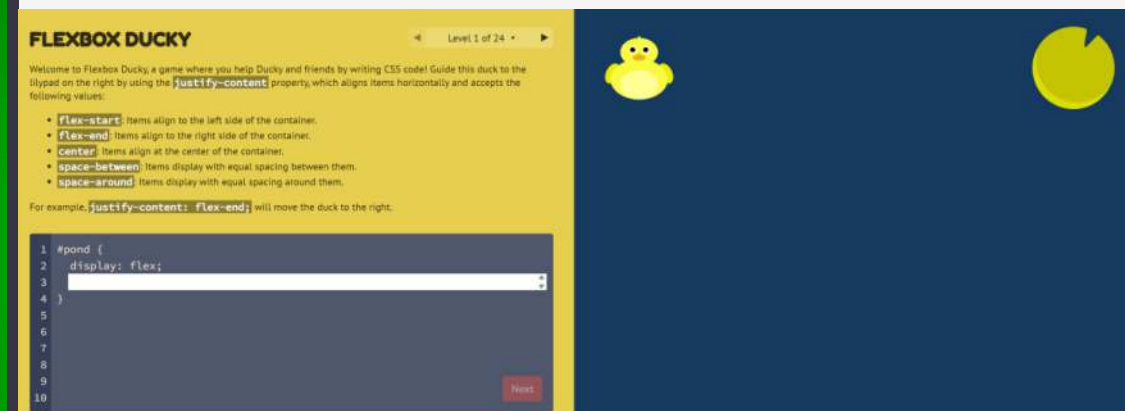
Flexbox Defense

<http://www.flexboxdefense.com/>



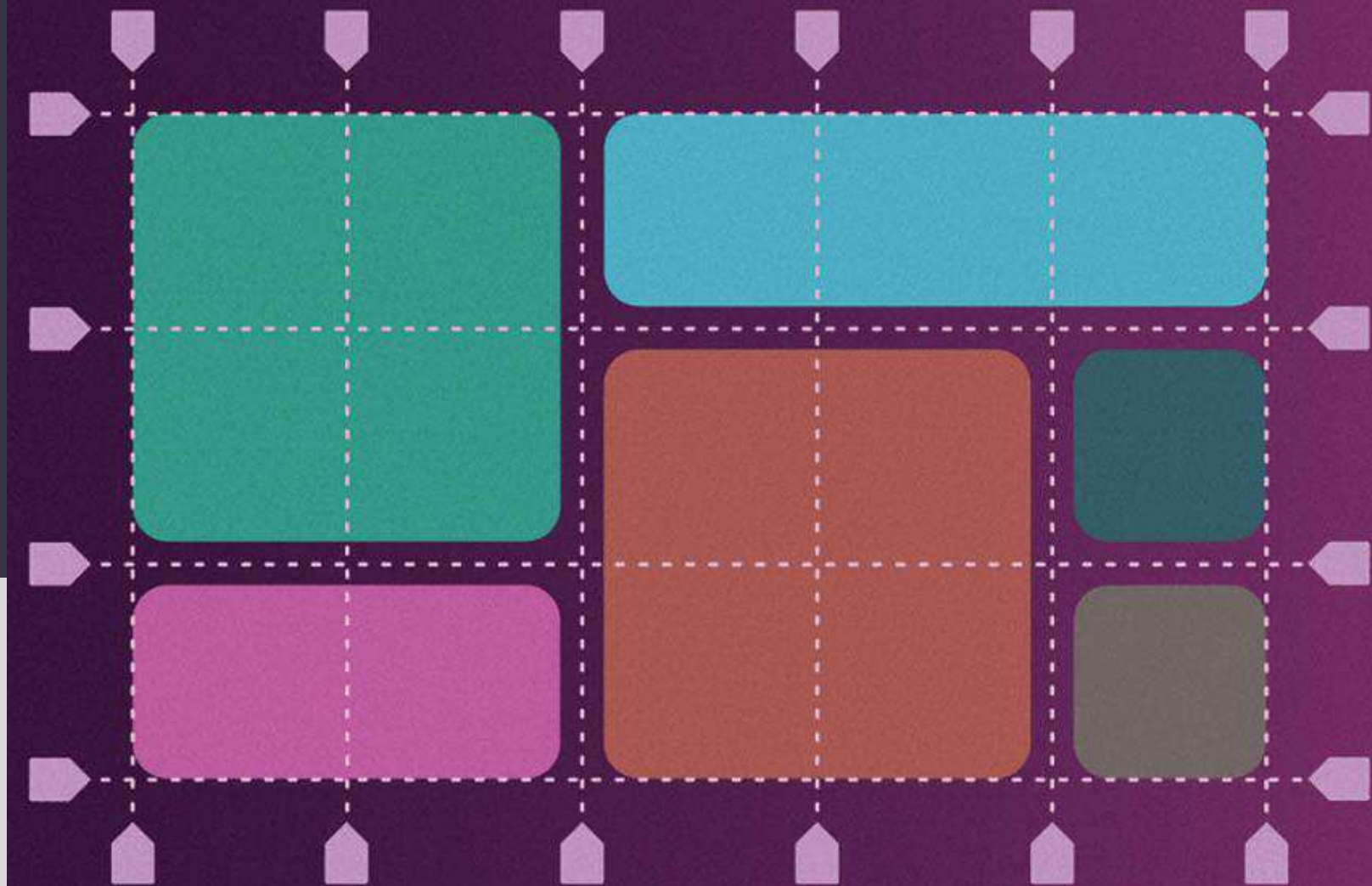
Flexbox Ducky

<https://courses.cs.washington.edu/courses/cse154/flexboxducky/>



3

GRID LAYOUT

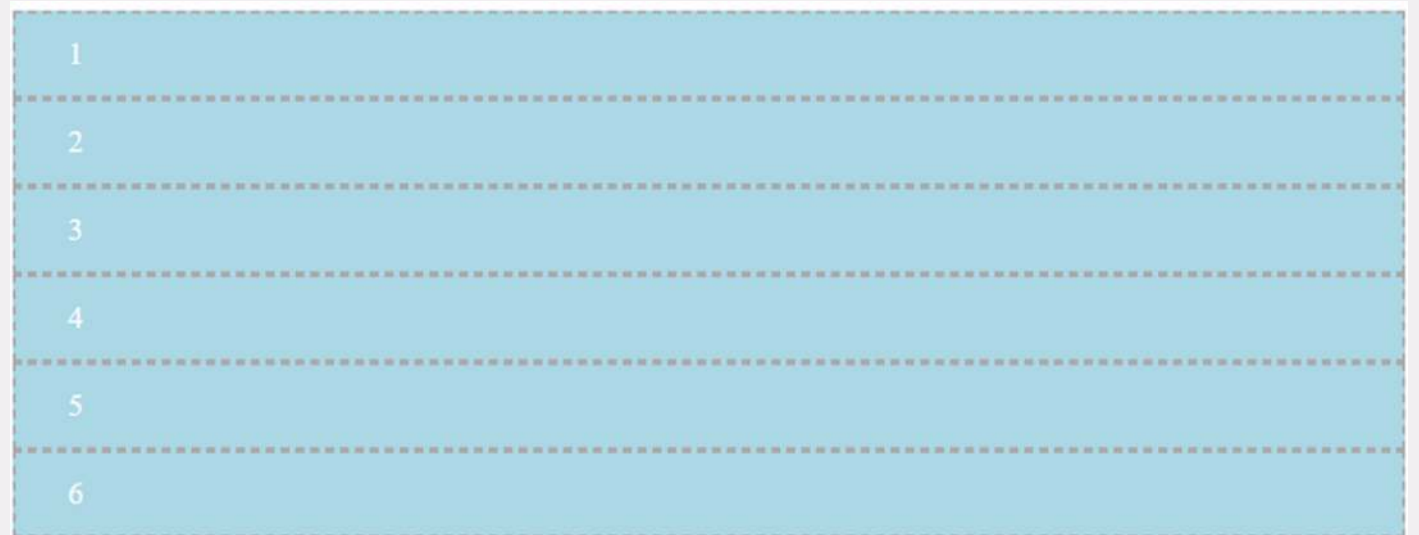
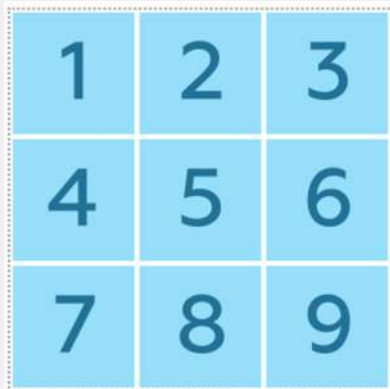


Grid Layout

CSS Grid Layout (далее просто Grid) — это способ двумерной раскладки. Именно ДВУмерной, в отличие от Flexbox. Flexbox позволяет полноценно управлять элементами только по одной оси.

Если элементу задано свойство **display** со значением **grid**, то такой элемент становится грид-контейнером. Дочерние элементы этого контейнера начинают подчиняться правилам грид-раскладки.

Снаружи грид-контейнер ведёт себя как блок

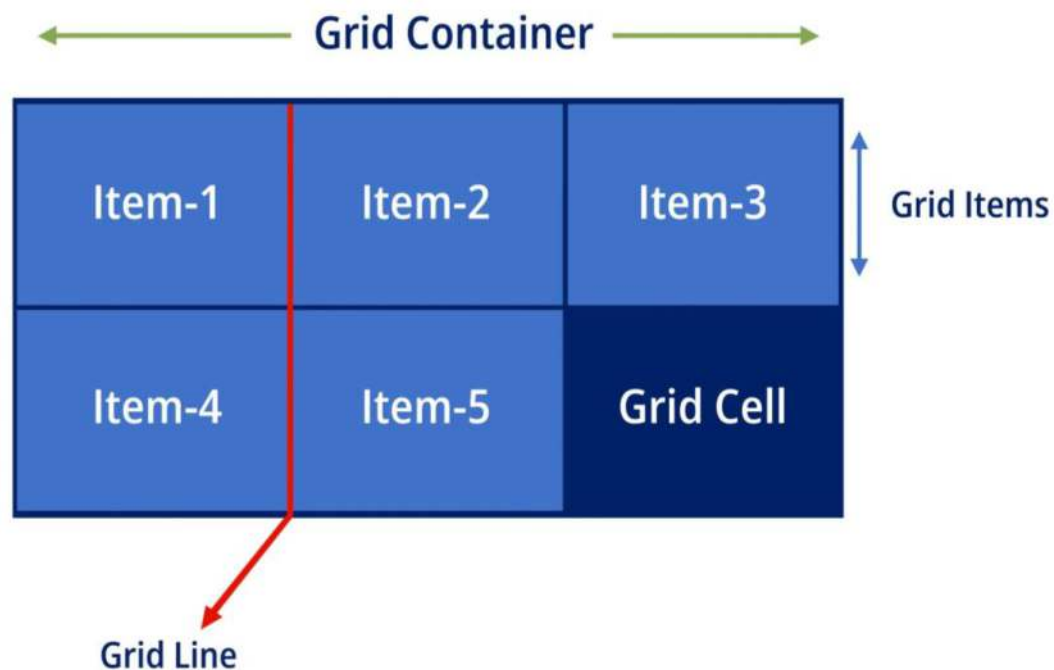


Основные термины

Грид-контейнер: родительский элемент, к которому применяется свойство `display: grid`.

Грид-линия: разделительная линия, формирующая структуру грида. Может быть как вертикальной (грид-линия колонки), так и горизонтальной (грид-линия ряда). Располагается по обе стороны от колонки или ряда. Используется для привязки грид-элементов.

Грид-элемент: дочерний элемент, прямой потомок грид-контейнера. Подчиняется правилам раскладки гридов.



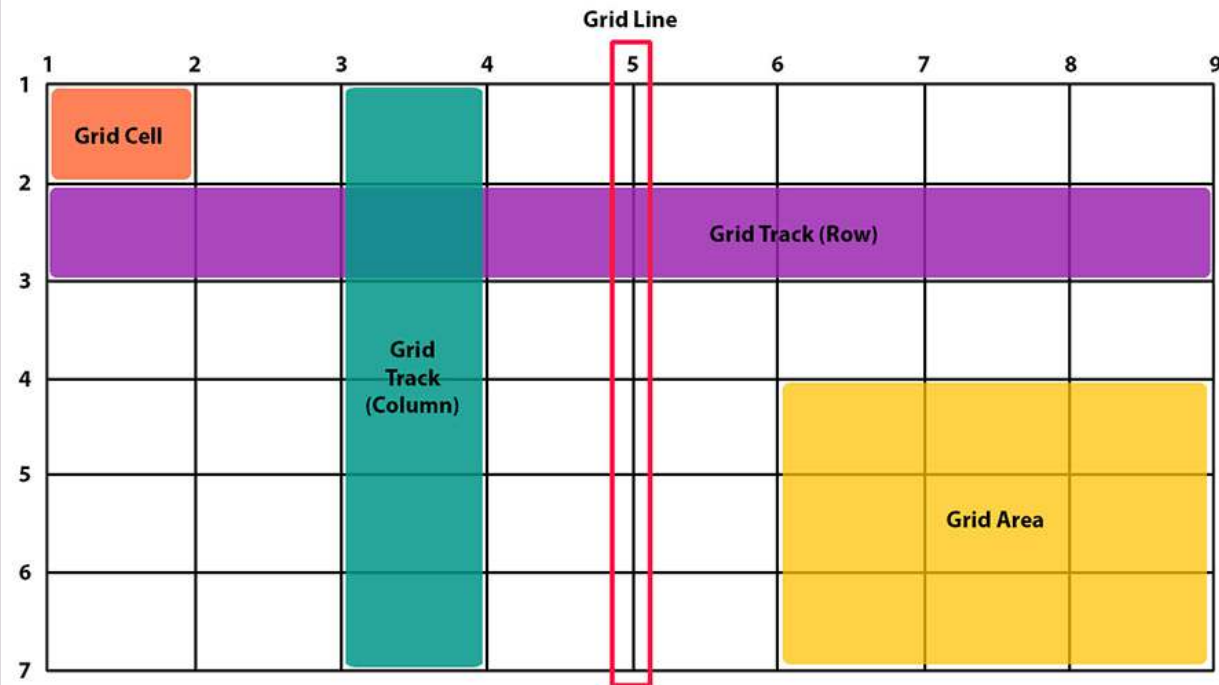
Основные термины

Грид-ячейка: пространство между соседними грид-линиями. Единица грид-сетки.

Грид-полоса: пространство между 2 соседними грид-линиями. Может быть проще думать о грид-полосе как о ряде или колонке

Грид-область: область, ограниченная четырьмя грид-линиями.

Может состоять из любого количества ячеек как по горизонтали, так и по вертикали



Grid Layout

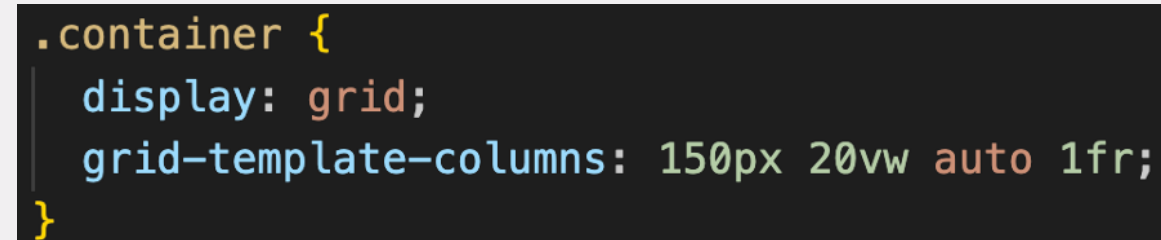
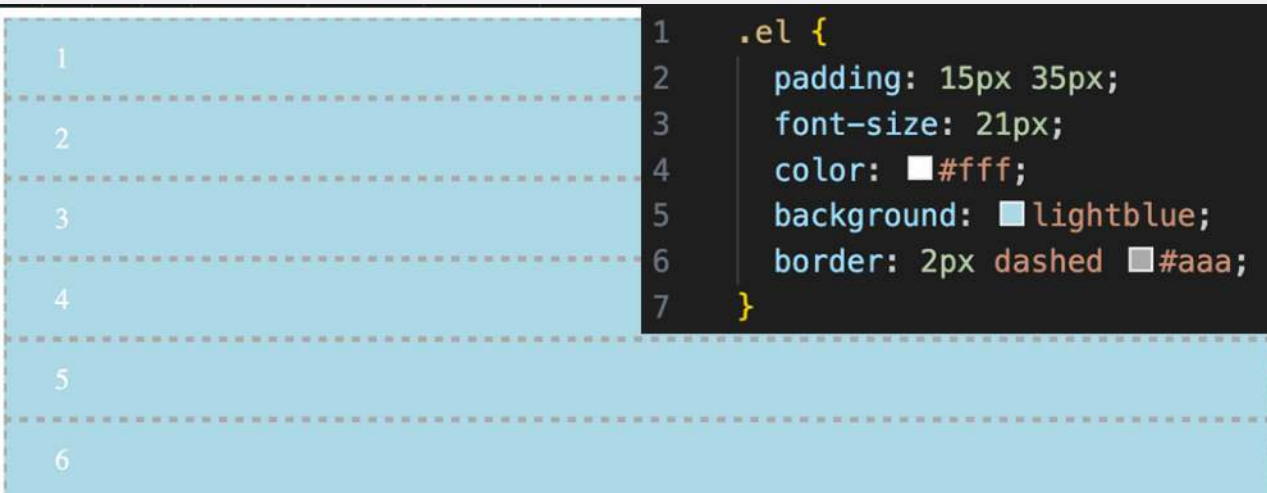
grid-template-columns определяет количество колонок и может задавать ширину каждой из них. В данном случае количество колонок равно 4, поскольку перечислены 4 параметра.

При этом ширина колонок может задаваться в разных единицах. Так **px** – это известные нам пиксели. **vw** - представляет собой процент ширины корневого элемента.

Один **VW** равен 1% ширины области просмотра.

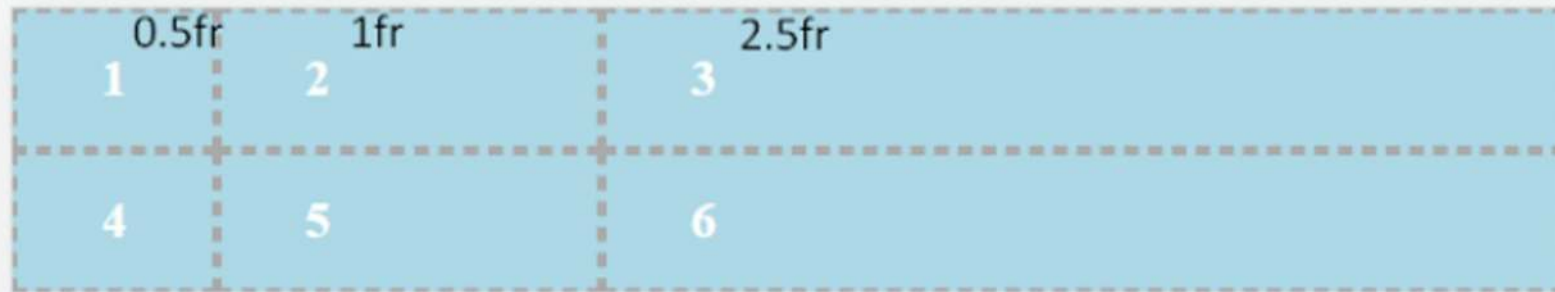
Третья колонка ужимается под контент (свойство **auto**).

А **1fr** здесь используется, чтоб занять всё оставшееся доступное место



Grid Layout

fr (fractional unit) всегда занимает свободное пространство. Когда несколько столбцов с ширинами в fr, цифрами мы указываем, какую часть свободного пространства должны делить между собой строки / колонки.



Общая доля свободного пространства (100%) будет равна сумме всех **fr**:

- ширина контейнера: $0.5fr + 1fr + 2.5fr = 4fr$ или 100%
- ширина первой колонки: $0.5fr / 4fr = 1/8$ или 12.5%
- ширина второй колонки: $1fr / 4fr = 1/4$ или 25%
- ширина третьей колонки: $2.5fr / 4fr = 5/8$ или 62,5%

Эту единицу измерения можно использовать даже с дробными значениями.

Например, строка **grid-template-columns: 0.5fr 1fr 2.5fr;** даст следующий результат:

grid-template-columns

Также работает и с grid-template-rows

1	2	3
4	5	6
7	8	9

```
.parent {  
  display: grid;  
  grid-template-columns: 50px 1fr 50px;  
}
```

1	2	3	
4	5	6	
7	8	9	

```
.parent {  
  display: grid;  
  grid-template-columns: 100px 50px 100px;  
}
```

1	2	3	
4	5	6	
7	8	9	

```
.parent {  
  display: grid;  
  grid-template-columns: repeat(3, 80px)  
}
```

grid-template-columns, grid-template-rows

Каждая линия может иметь больше одного имени.

Это чем-то похоже на классы в HTML (можно задать элементу больше одного класса).

```
.parent {  
  display: grid;  
  grid-template-columns: [start] 250px [line2] 400px [line3] 600px [end];  
  grid-template-rows: [row1-start] 15rem [row1-end] 30vh [last];  
}
```



При работ с данной разметкой можно явно именовать грид-линии, используя для этого квадратные скобки

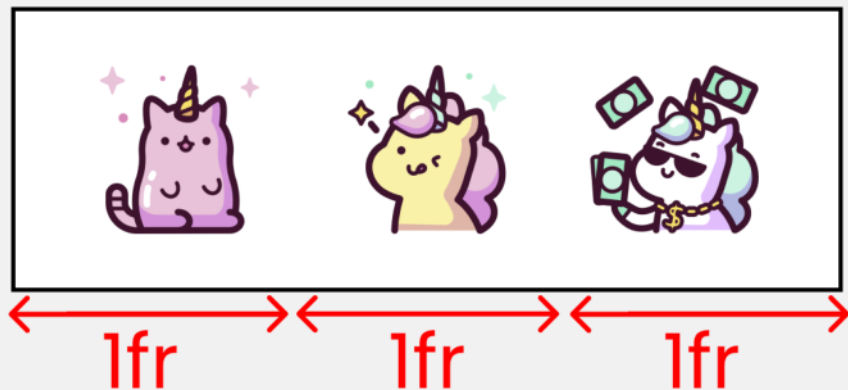
repeat()

Если нужны одинаковые колонки или ряды, то можно воспользоваться функцией **repeat()**.

Так будет создано 3 колонки по 250 пикселей

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 250px);  
}
```

grid-template-columns : repeat(3, 1fr);



fr

fr (от fraction = доля, часть) отвечает за свободное пространство внутри грид-контейнера.

Например, этот код создаст три колонки, каждая из которых будет занимать 1/3 ширины родителя

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

auto-fill

В тех случаях, когда количество колонок или строк не известно для свойств **grid-template-columns** и **grid-template-rows** можно установить значения **auto-fill** или **auto-fit** в формате

```
.container {  
  grid-template-columns: repeat(/* auto-fill или auto-fit, размер колонки */);  
}
```

auto-fill стремится заполнить колонками всё доступное пространство, а когда элементы заканчиваются, создаёт пустые колонки, равномерно распределяя доступную область между существующими и «виртуальными» колонками

auto-fit

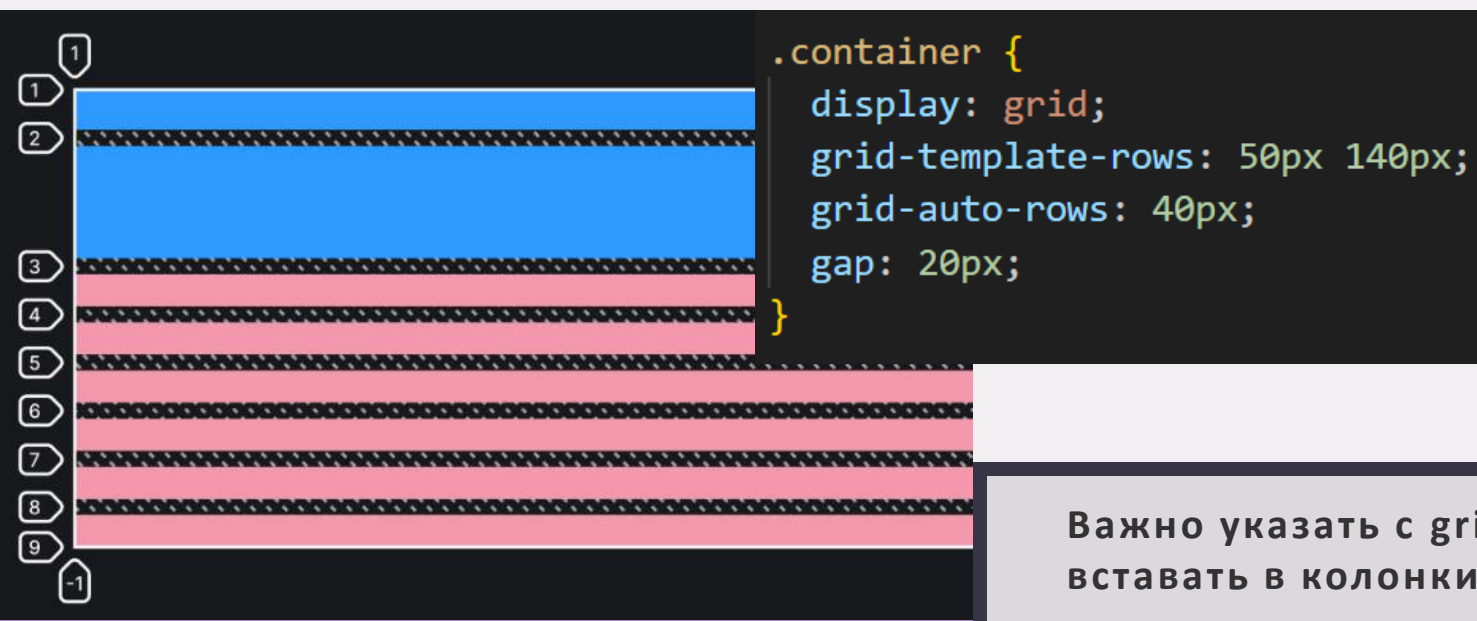
Оба эти параметра (**auto-fill** и **auto-fit**) автоматически добавляют в грид-сетку из примера выше колонки. Но эти колонки ведут себя немного по-разному

auto-fit действует похожим образом, но, схлопывает пустые колонки и отдаёт больше места под заполненные

grid-auto-columns, grid-auto-rows

Если элементов внутри грид-контейнера больше, чем может поместиться в заданные явно ряды и колонки, то для них создаются автоматические, неявные ряды и колонки. При помощи свойств **grid-auto-columns** и **grid-auto-rows** можно управлять размерами этих автоматических рядов и колонок.

Можно задавать больше одного значения для автоматических колонок или рядов. Тогда паттерн размера будет повторяться до тех пор, пока не кончатся грид-элементы.



Важно указать с **grid-auto-flow: column**, что элементы должны вставать в колонки, чтобы элементы без контента были видны.

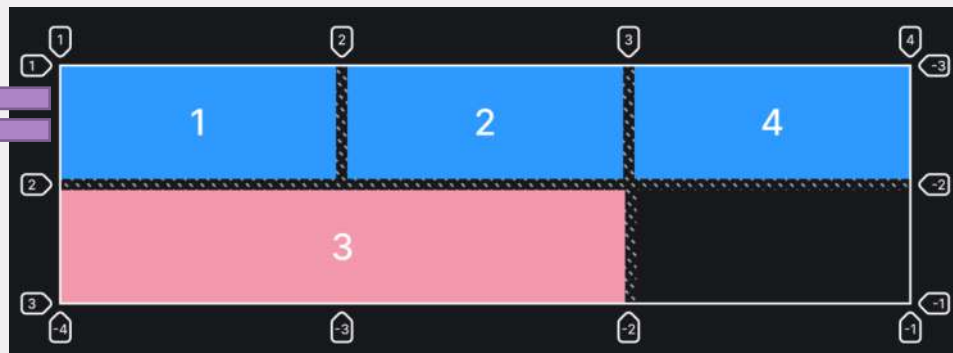
grid-auto-flow

Если грид-элементов больше, чем явно объявленных колонок или рядов, то они автоматически размещаются внутри родителя. А вот каким образом можно указать при помощи **grid-auto-flow**. По умолчанию значение у этого свойства **row**, лишние элементы будут выстраиваться в ряды в рамках явно заданных колонок. Возможные значения: **row**, **column** и **dense** (браузер старается заполнить дырки в разметке, если размеры элементов позволяют).

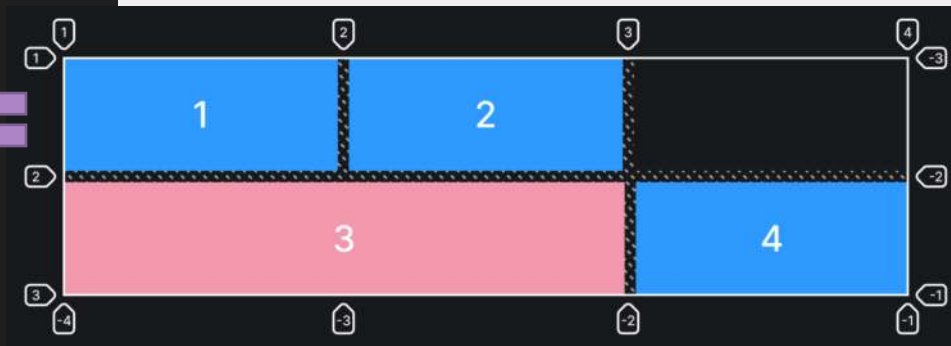
```
.container {  
  display: grid;  
  /* Три колонки */  
  grid-template-columns: auto auto auto;  
  /* Два ряда */  
  grid-template-rows: auto auto;  
  /* Автоматическое размещение в ряд */  
  grid-auto-flow: row;  
  /* Отступы между ячейками */  
  gap: 20px;  
}
```



```
/* Автоматическое размещение в ряд */  
grid-auto-flow: row dense;  
}
```



```
.item3 {  
  /* Занимает один ряд и  
  растягивается на две колонки */  
  grid-column: span 2;  
}
```



grid-template-areas

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 500px);  
  grid-template-rows: repeat(4, 1fr);  
  grid-template-areas:  
    "header header header"  
    "content content 🐙"  
    "content content ."  
    "footer footer footer";  
}
```

```
.item1 {  
  grid-area: header;  
}
```

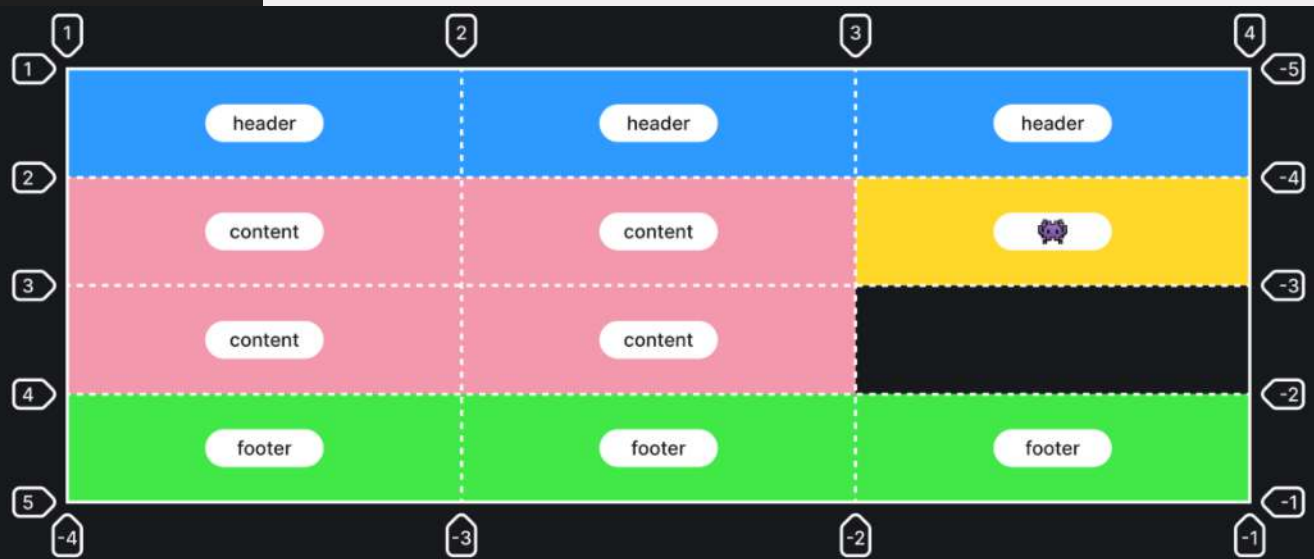
```
.item2 {  
  grid-area: content;  
}
```

```
.item3 {  
  grid-area: 🐙;  
}
```

```
.item4 {  
  grid-area: footer;  
}
```

Позволяет задать шаблон сетки расположения элементов внутри грид-контейнера. Имена областей задаются при помощи свойства **grid-area**. Текущее свойство **grid-template-areas** просто указывает, где должны располагаться эти грид-области. Возможные значения:

- **none** (значение по умолчанию) — области сетки не задано имя.
- **.** — означает пустую ячейку.
- **название** — название области, любое словом или даже эмодзи! 🐙



Нужно называть каждую из ячеек. Так, если шапка сайта будет занимать 3 существующие колонки, то нужно будет трижды написать название области

grid-template

```
.container {  
  display: grid;  
  grid-template:  
    [row1-start] 25px [row1-end]  
    [row2-start] 25px [row2-end]  
    / auto 50px auto;  
  grid-template-areas:  
    "header header header"  
    "footer footer footer";  
}
```

```
.container {  
  display: grid;  
  grid-template: repeat(4, 1fr) / repeat(3, 500px);  
}
```

Свойство-шорткат для **grid-template-rows** и **grid-template-columns**.

Позволяет записать все значения в одну строку.

Главное после этого не запутаться в них...

Можно прописать все колонки и ряды сразу, разделяя их слэшем /.

Сперва идут ряды, а затем колонки, не перепутайте!

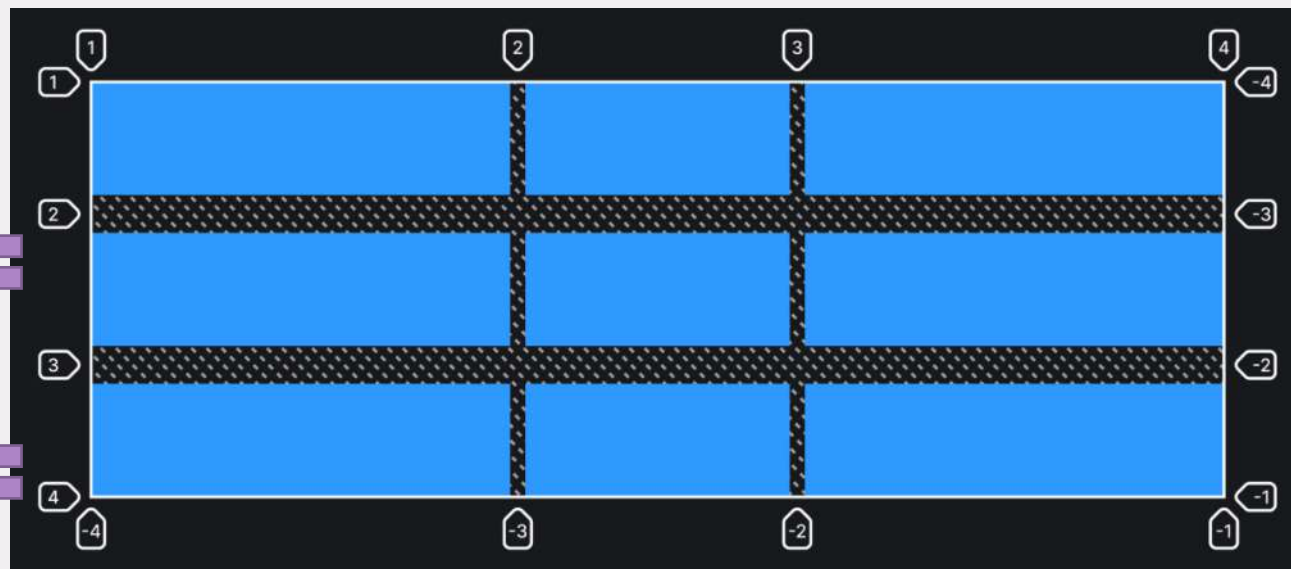
row-gap, column-gap, gap

`row-gap`, `column-gap` задают отступы между рядами или колонками.

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 350px 1fr;  
  grid-template-rows: repeat(3, 150px);  
  /* Отступы между рядами */  
  row-gap: 50px;  
  /* Отступы между колонками */  
  column-gap: 20px;  
}
```

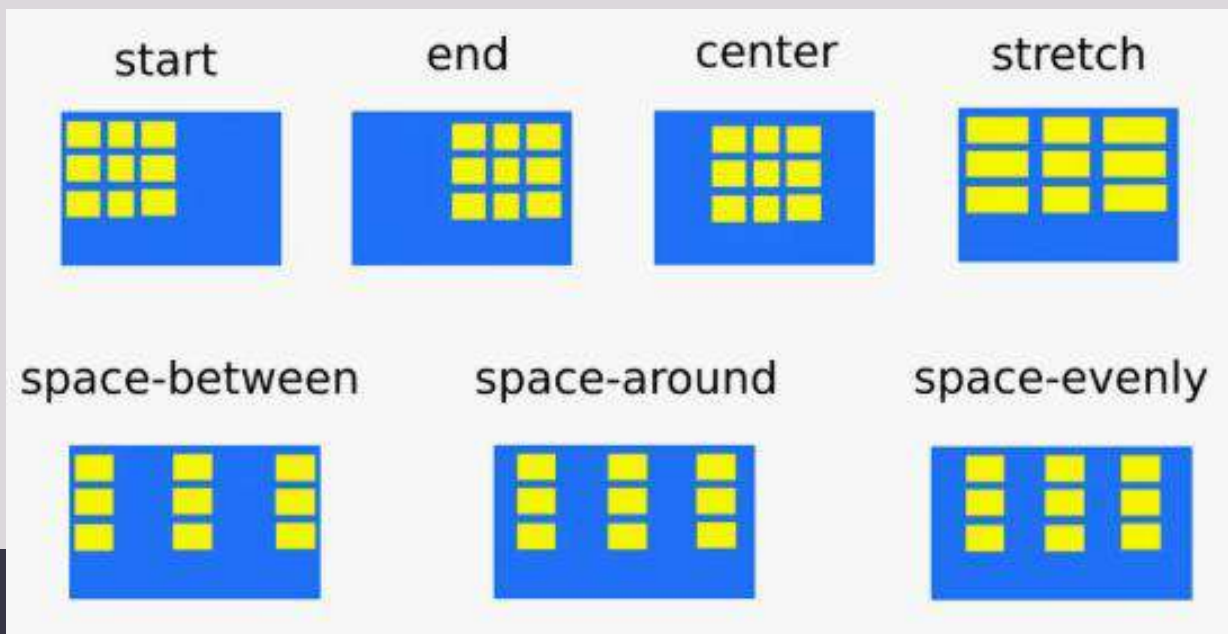
```
.container {  
  display: grid;  
  grid-template-columns: 1fr 350px 1fr;  
  grid-template-rows: repeat(3, 150px);  
  gap: 50px 20px;  
}
```

`gap` – шорткат для записи значений свойств `row-gap` и `column-gap`. Значения разделяются пробелом.



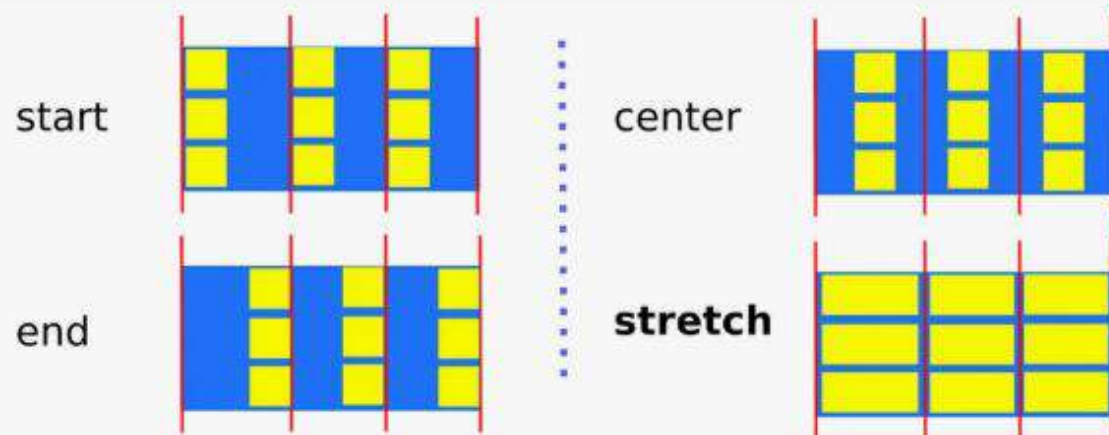
justify-content

Свойство, с помощью которого можно выровнять элементы вдоль оси строки. Работает, только если общая ширина столбцов меньше ширины контейнера сетки. Необходимо свободное пространство вдоль оси строки контейнера, чтобы выровнять его столбцы слева или справа.



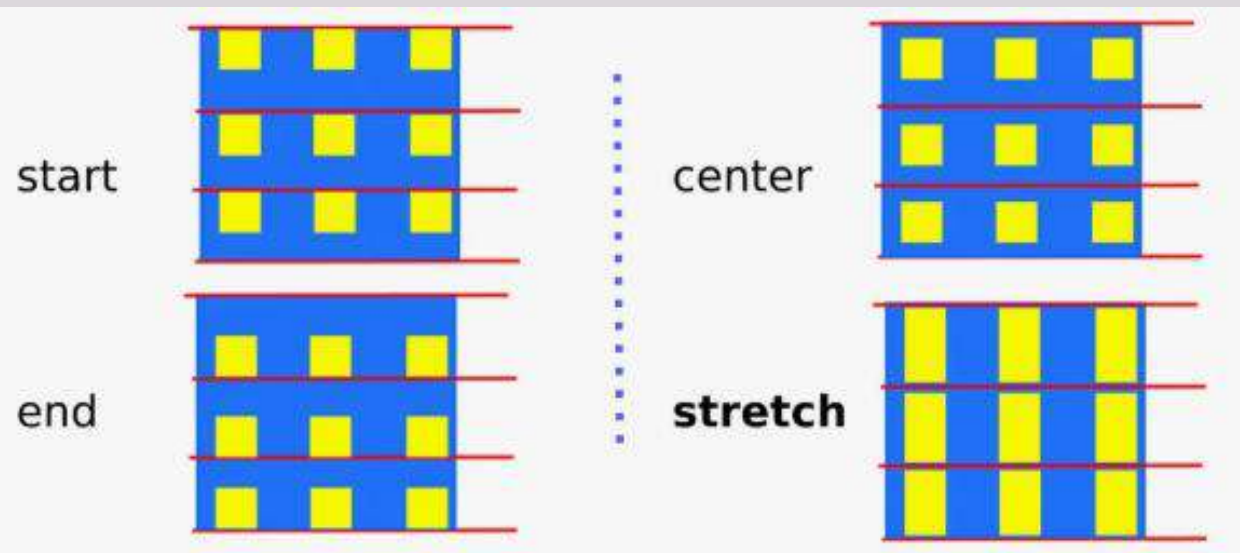
justify-items

Свойство, с помощью которого задаётся выравнивание элементов по горизонтальной оси. Применяется ко всем элементам внутри грид-родителя.



align-items

Свойство, с помощью которого можно выровнять элементы по вертикальной оси внутри грид-контейнера.

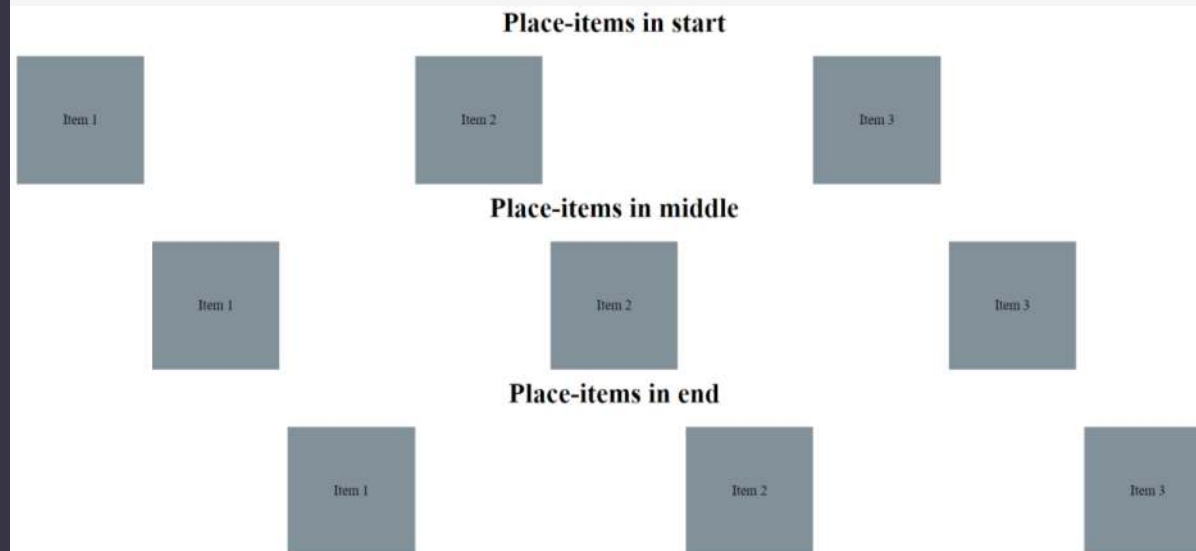


place-items

Шорткат для указания значений сразу и для **align-items** и для **justify-items**.

Указывать нужно именно в таком порядке.

```
1 .item {  
2   display: grid; /* or flex */  
3   place-items: <align-items> <justify-items>;  
4 }
```



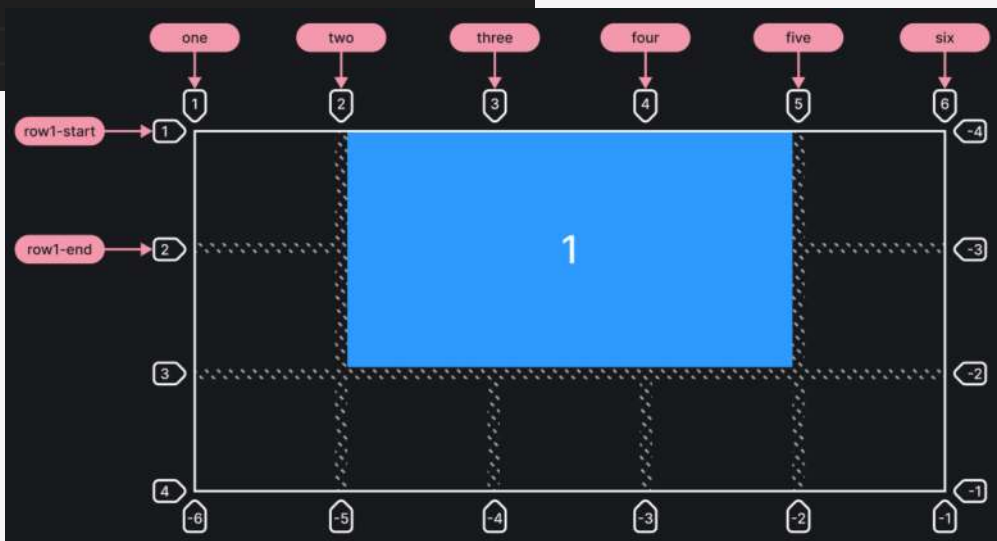
grid-column-start, grid-column-end, grid-row-start, grid-row-end

```
.container {  
  display: grid;  
  grid-template-columns: [one] 1fr [two] 1fr [three] 1fr  
                        [four] 1fr [five] 1fr [six];  
  grid-template-rows: [row1-start] 1fr [row1-end] 1fr 1fr;  
}
```

```
.item1 {  
  grid-column-start: 2;  
  grid-column-end: five;  
  grid-row-start: row1-start;  
  grid-row-end: 3;  
}
```

Определяют положение элемента внутри грид-сетки при помощи указания на конкретные направляющие линии. Возможные значения:

- **название или номер линии** – порядковый номер или название конкретной линии.
- **span число** – элемент растянется на указанное количество ячеек.
- **span имя** – элемент будет растягиваться до следующей указанной линии.
- **auto** – автоматическое размещение, автоматический диапазон клеток или дефолтное растягивание элемента, равное одному.



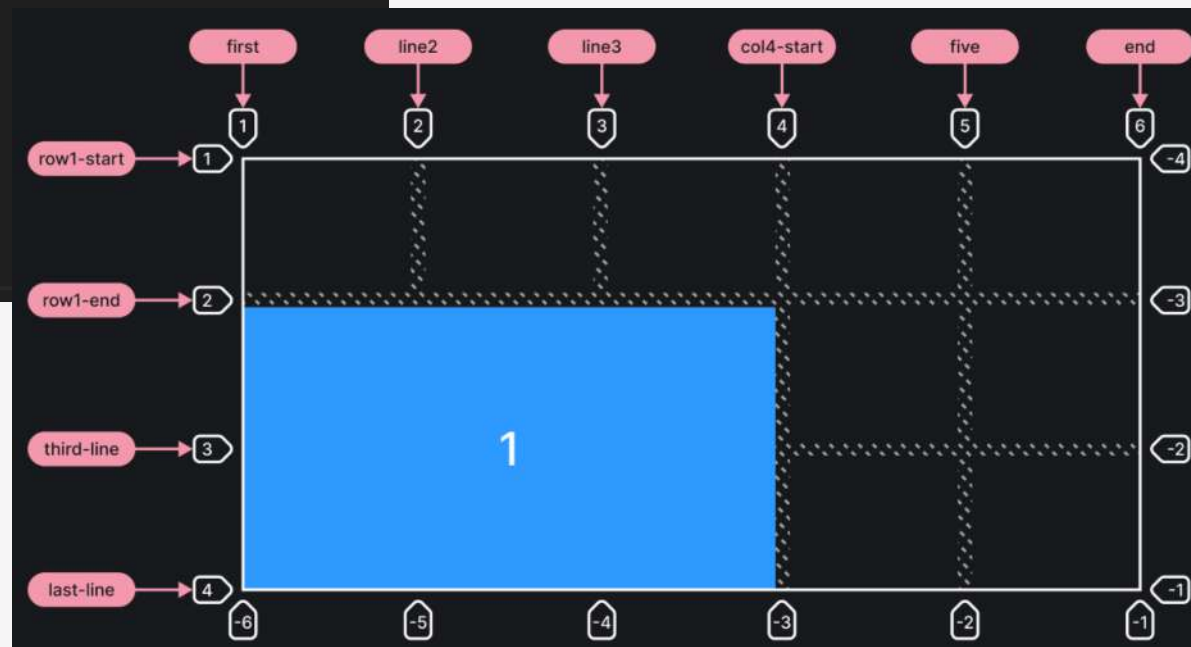
Свойства грид-элементов

grid-column-start, grid-column-end, grid-row-start, grid-row-end

```
.container {  
  display: grid;  
  grid-template-columns: [first] 1fr [line2] 1fr [line3] 1fr [col4-start] 1fr [five] 1fr [end];  
  grid-template-rows: [row1-start] 1fr [row1-end] 1fr [third-line] 1fr [last-line];  
}
```

```
.item1 {  
  grid-column-start: 1;  
  grid-column-end: span col4-start;  
  grid-row-start: 2;  
  grid-row-end: span 2;  
}
```

Определяют положение элемента внутри грид-сетки при помощи указания на конкретные направляющие линии.
Возможные значения:



grid-column, grid-row

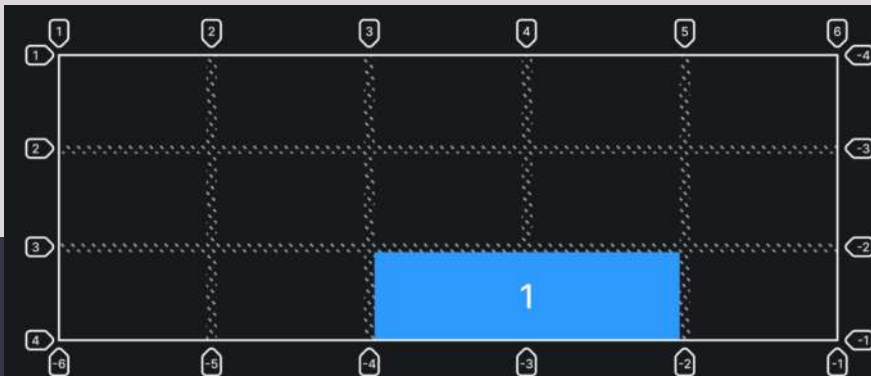
Шорткаты для **grid-column-start**, **grid-column-end** и **grid-row-start**, **grid-row-end** соответственно.

Значения для ***-start** и ***-end** разделяются слэшем.

Можно использовать ключевое слово **span**, буквально говорящее «растянись на ___».

Элемент начинается с линии 3 по горизонтали, растягивается на 2 ячейки. По вертикали элемент начинается с линии 3 и заканчивается у 4-й линии. Если опустить слэш и 2-е значение, то элемент будет размером в одну ячейку.

```
.item1 {  
  grid-column: 3 / span 2;  
  grid-row: 3 / 4;  
}
```



grid-area

Двуличное свойство, которое может и указывать элементу, какую из именованных областей ему нужно занять, и служить шорткатом для одновременного указания значений для четырёх свойств: **grid-column-start**, **grid-column-end**, **grid-row-start** и **grid-row-end**

```
.item1 {  
  /* Займёт область content внутри грид-сетки */  
  grid-area: content;  
}
```

Пример с указанием всех 4-х значений в строку. При таком указании, значения указываются в порядке: **row-start / column-start / row-end / column-end**, сначала оба начала, потом оба конца.

```
.item2 {  
  grid-area: 1 / col4-start / last-line / 6;  
}
```

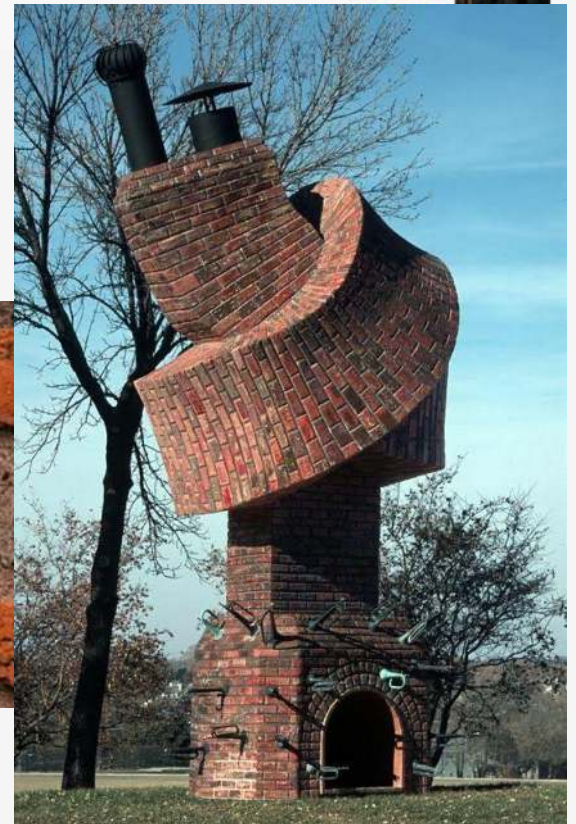
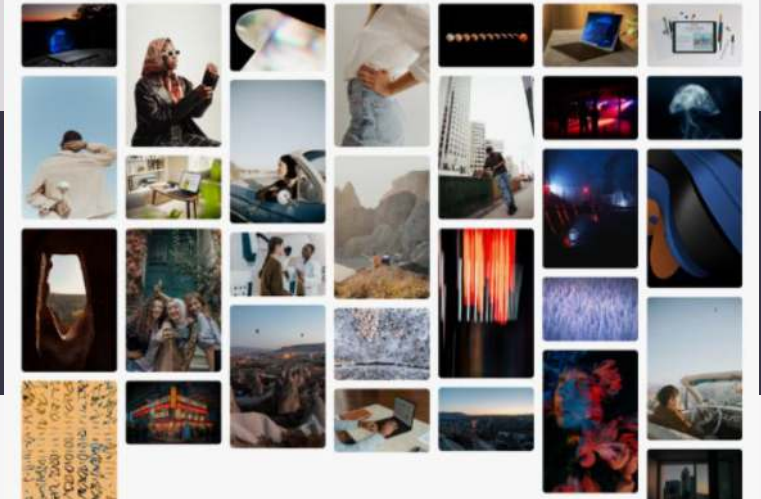
4

MASONRY LAYOUT

Галерея, как из pinterest

Masonry layout

Масонри-макет (Masonry layout) — это популярный дизайн веб-страниц, который позволяет размещать элементы контента в виде сетки с переменной высотой столбцов. Он часто используется для отображения изображений, фотогалерей или карточек контента.



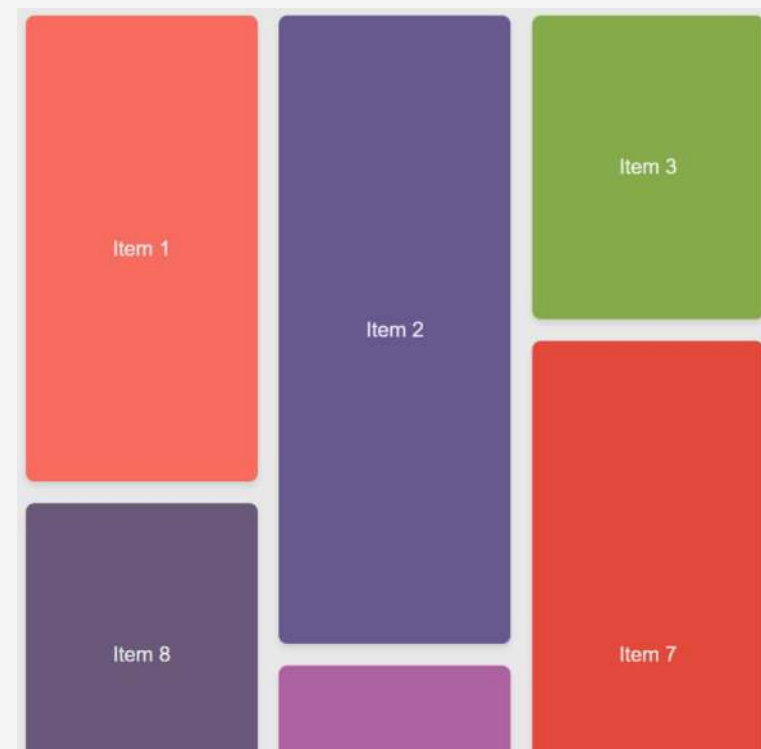
<https://reintech.io/blog/creating-a-masonry-layout-with-css>

Вариант 1 – экспериментальная технология grid

Уровень 3 спецификации CSS-разметки включает значения `masonry` для **grid-template-columns** и **grid-template-rows**. Чтобы создать компоновку, в которой столбцы являются осью сетки, а строки — осью компоновки, используется **grid-template-columns** и **grid-template-rows**. Дочерние элементы будут размещаться по строкам, как при обычном размещении в сетке.

По мере перемещения элементов в новые строки они будут отображаться в соответствии с алгоритмом кладки.

Элементы будут загружаться в столбец, в котором больше всего свободного места, что обеспечит плотную компоновку без строгих границ между рядами



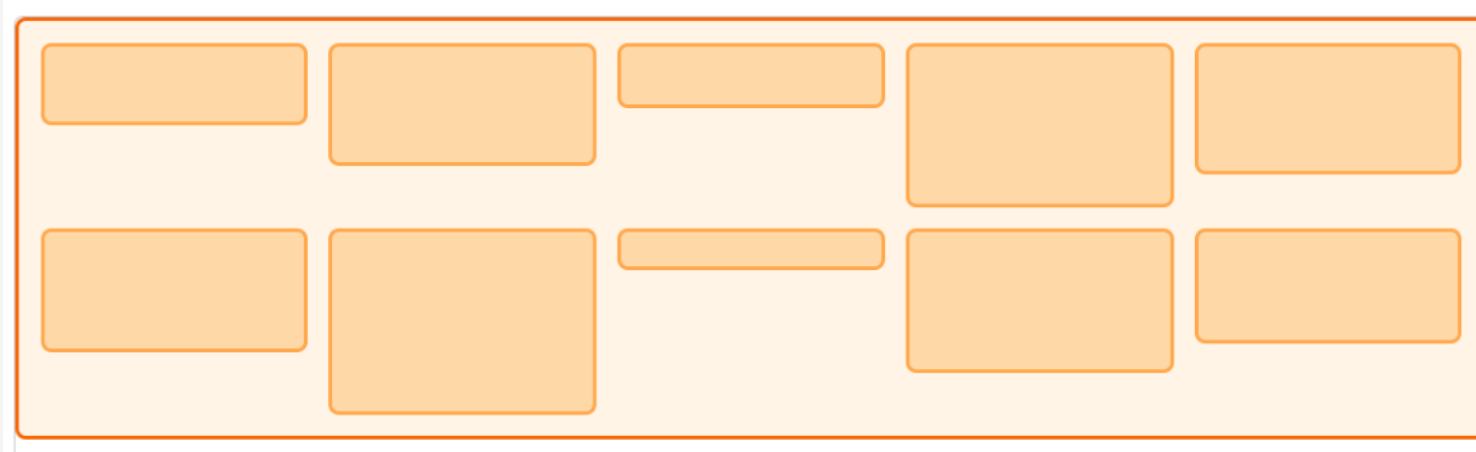
Masonry layout

Вариант 1 – экспериментальная технология grid

```
.grid {  
  display: grid;  
  gap: 10px;  
  grid-template-columns: repeat(auto-fill, minmax(120px, 1fr));  
  grid-template-rows: masonry;  
}
```

```
<div class="grid">  
  <div class="item"></div>  
  <div class="item"></div>  
  <div class="item"></div>  
  <div class="item"></div>  
</div>
```

```
// prettier-ignore  
const itemSizes = [  
  "2em", "3em", "1.6em", "4em", "3.2em",  
  "3em", "4.5em", "1em", "3.5em", "2.8em",  
];  
const items = document.querySelectorAll(".item");  
for (let i = 0; i < items.length; i++) {  
  items[i].style.blockSize = itemSizes[i];  
}
```



Вариант 1 – экспериментальная технология grid

При работе рекомендуется ориентироваться на таблицу совместимости технологии. Во многих браузерах на 7 ноября 2025 года она не поддерживается. В них вместо этого будет использоваться обычное автоматическое размещение сетки

												
	 Chrome	 Edge	 Firefox	 Opera	 Safari	 Chrome Android	 Firefox for Android	 Opera Android	 Safari on iOS	 Samsung Internet	 WebView Android	 WebView on iOS
masonry 	 No	 No	 77	 No	 TP	 No	 No	 No	 No	 No	 No	 No
	*	*	 *	*	*	*		*		*	*	

Masonry layout

Вариант 2 – работаем с колонками

```
<main class="container">
  <section class="masonry">
    <!-- Новость 1 -->
    <article class="masonry-item">...
  </article>

    <!-- Новость 2 -->
    <article class="masonry-item">...
  </article>

    <!-- Новость 3 -->
    <article class="masonry-item">...
  </article>

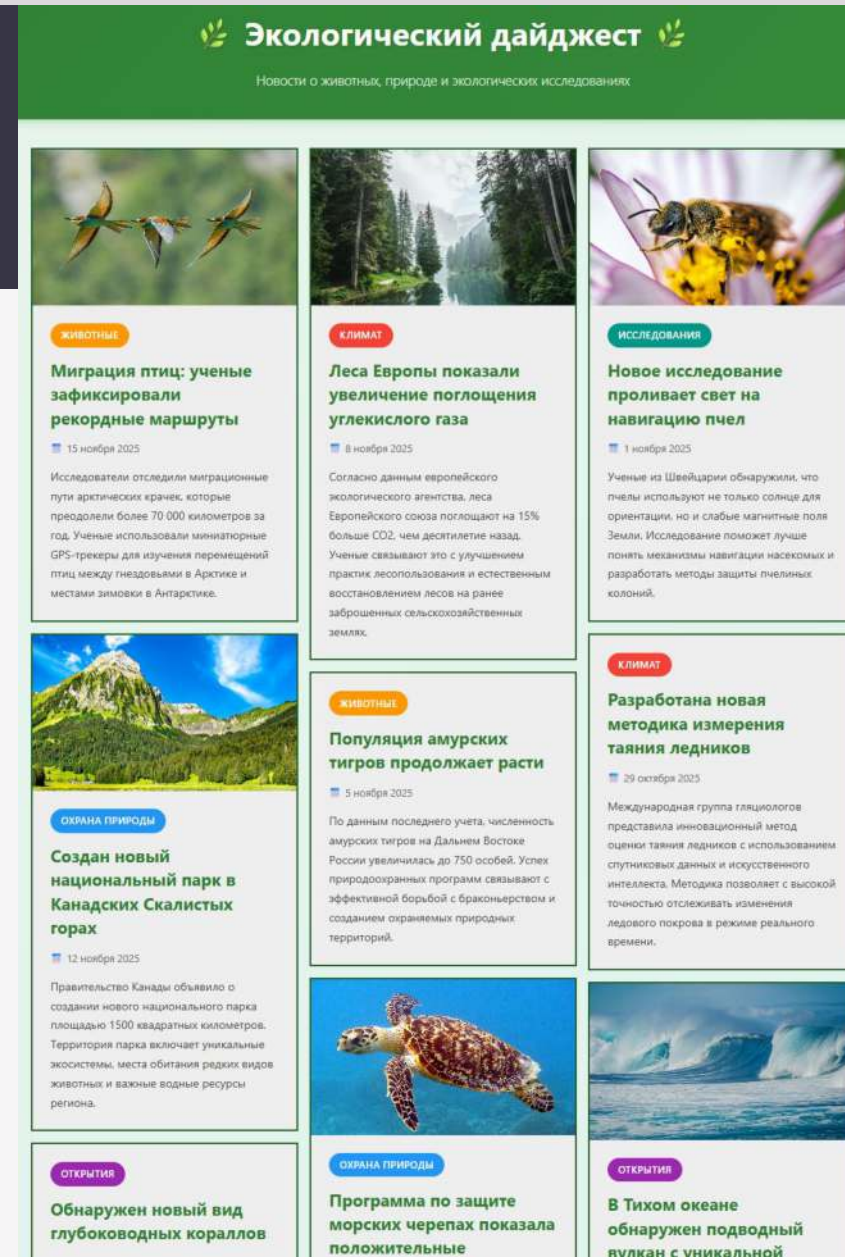
    <!-- Новость 4 -->
    <article class="masonry-item">...
  </article>

    <!-- Новость 5 -->
    <article class="masonry-item">
      <div class="news-content">
```



```
.masonry {
  column-count: 3;
  column-gap: 1em;
}

.masonry-item {
  display: inline-block;
  margin: 0 0 1em;
  width: 100%;
}
```



<https://masonry.desandro.com/#install>

<https://github.com/desandro/masonry>

Вариант 3 – одноименная библиотека

Masonry полезна для сайтов с большим объёмом контента, например, галерей, портфолио и витрин продуктов.

Библиотека размещает элементы на основе доступного вертикального пространства, создавая адаптивный макет.

Masonry

Layout

Options

Methods

Events

Extras

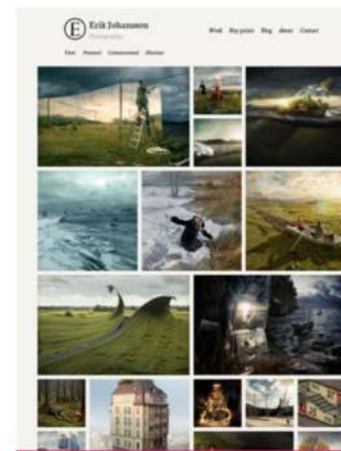
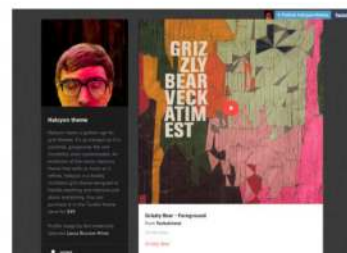
FAQ

Masonry

Cascading grid layout library

What is Masonry?

Masonry is a JavaScript grid layout library. It works by placing elements in optimal position based on available vertical space, sort of like a mason fitting stones in a wall. You've probably seen it in use all over the Internet.



Erik Johansson



Download masonry.
pkgd.min.js



Download these docs



Masonry on GitHub



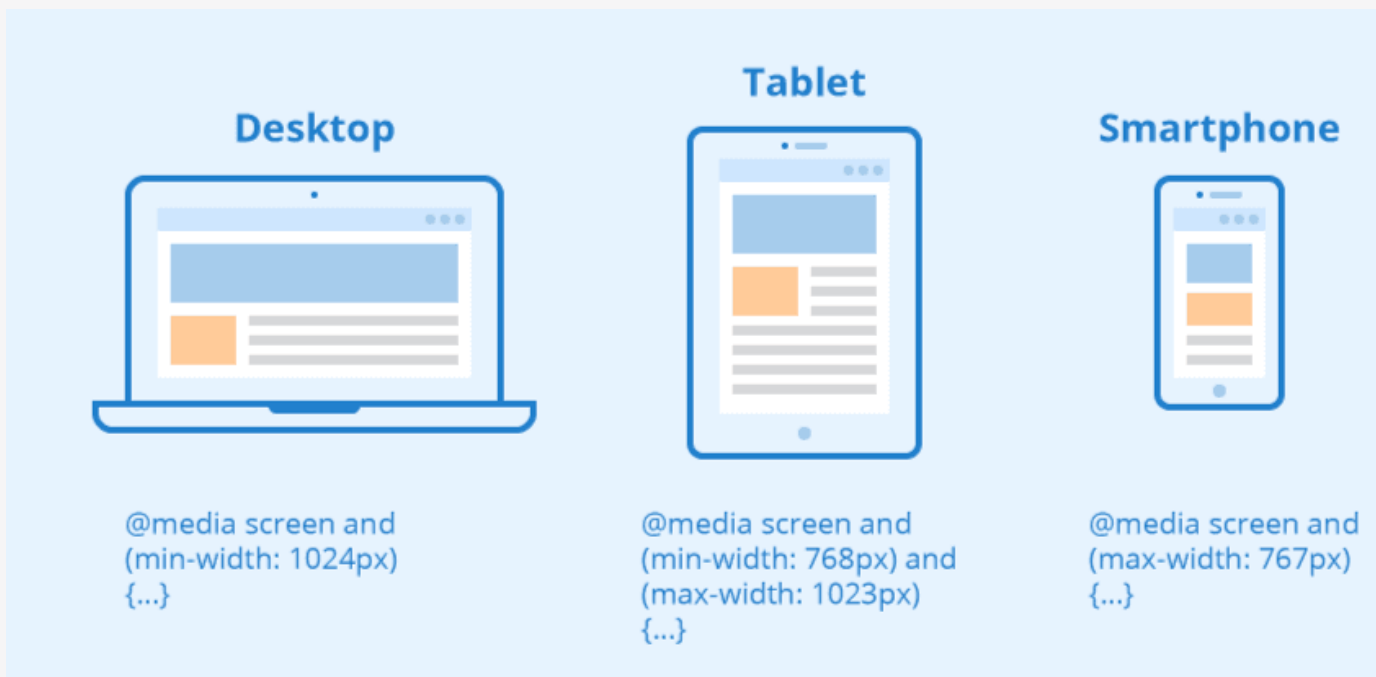
Beyoncé | I Am

5

@MEDIA/МЕДИАЗАПРОСЫ

При вёрстке адаптивных сайтов часто нужно, чтобы какой-то набор стилей применился только при соблюдении определённых условий. Например, текст должен стать зелёным при горизонтальной ориентации смартфона. Или блок займёт всю ширину родителя, если ширина экрана больше или равна 1500 пикселям.

Для подобных случаев и существуют **@media**. Они помогают адаптировать вёрстку под разные экраны и устройства и при этом не писать лишний код.



@media

```
@media [тип устройства] [характеристика устройства] {  
    /* CSS-правила */  
}
```

Директива, которая позволяет задавать разные стили для разных параметров экрана (ширины, высоты, ориентации и даже типа устройства).

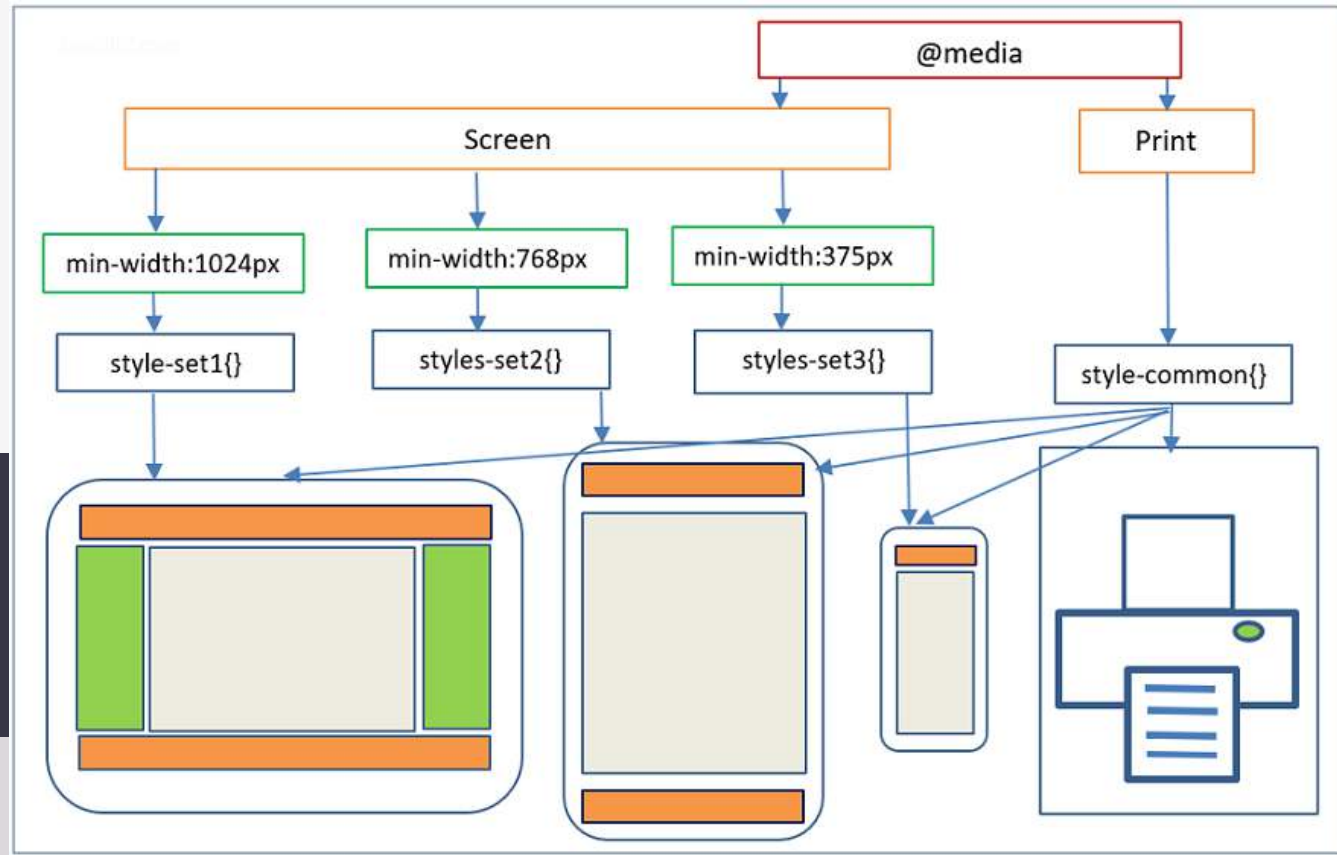
Есть три типа устройств, которые можем указать:

- **all** — медиавыражение применится ко всем устройствам.
Если не задать никакой тип, по умолчанию применится этот;
- **print** — стили внутри такого медиавыражения применяются при печати на принтерах или экспорте в PDF, в том числе в режиме предпросмотра документа;
- **screen** — для устройств с экранами.

В большинстве случаев, когда пишете стили только для экрана, указывать типы **screen** или **all** не нужно. Реальное практическое использование есть только у **print**.

Типы устройств

Медиазапросы / @media



Характеристики устройства

Экран современных устройств может быть разделён на несколько частей при помощи складки или шарнира между экранами. Отдельные области экрана называют **сегментами**

horizontal-viewport-segments

проверка на сколько сегментов по горизонтали разделён экран.

vertical-viewport-segments

проверка на сколько сегментов по вертикали разделён экран.

Значение: положительное целое число

```
@media (horizontal-viewport-segments: 1) { ... }
```

```
@media (overflow-block: paged) { ... }
```

overflow-block

проверяет, как устройство поступает с контентом, непоместившимся в экран по блочной оси. Значения (none, scroll, optional-paged, paged)

overflow-inline

проверяет, можно ли прокручивать содержимое, выходящее за пределы области просмотра по строчной оси. Значения (none, scroll)

```
@media (overflow-inline: scroll) { ... }
```

Характеристики устройства

Экран современных устройств может быть разделён на несколько частей при помощи складки или шарнира между экранами. Отдельные области экрана называют **сегментами**

grid

проверяет, является ли экран растровым (все современные экраны) или сеточным (старые телефоны или текстовые терминалы). Значения: 0 для растровых экранов и 1 для сеточных экранов.

```
@media (grid: 1) { ... }
```

resolution, min-resolution,
max-resolution

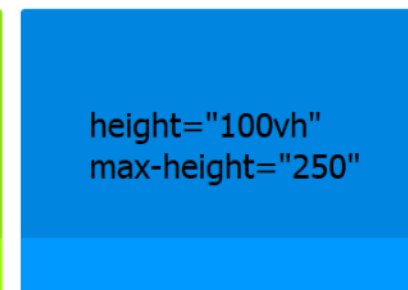
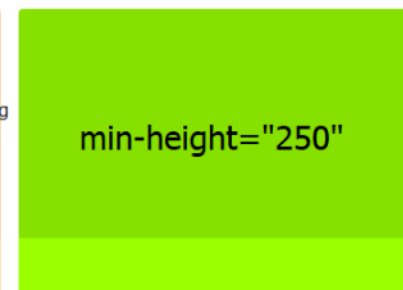
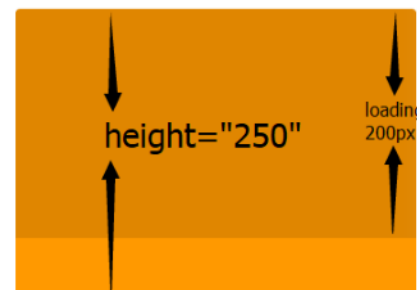
проверяет разрешение устройства в dpi или dpcm.

Значение: число с нужными единицами измерения. Единицы измерения для разрешения устройства.

```
@media (max-resolution: 300dpi) { ... }
```

Характеристики страницы и окна браузера

height, min-height, max-height
проверка высоты окна браузера, минимальной или максимальной высот. Значение: положительное целое или дробное число.



```
@media (min-height: 30rem) { ... }
```

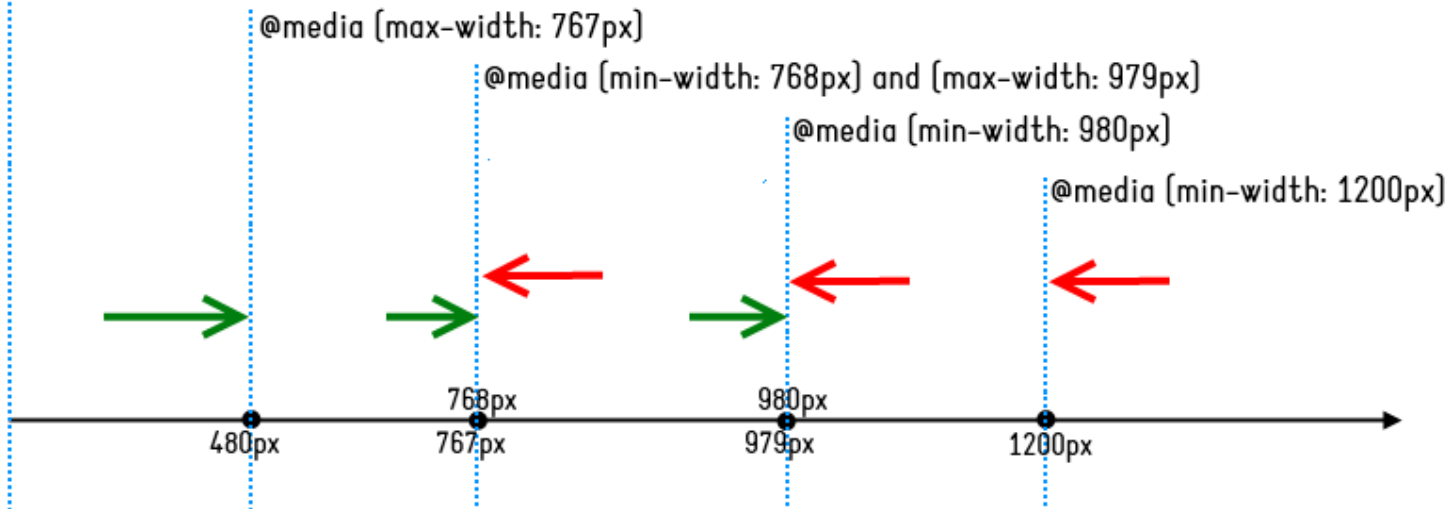
@media (max-width: 480px)

@media (max-width: 767px)

@media (min-width: 768px) and (max-width: 979px)

@media (min-width: 980px)

@media (min-width: 1200px)



width, min-width, max-width
проверка ширины окна браузера, минимальной или максимальной ширин. Значение: положительное целое или дробное число с любыми доступными единицами измерения.

```
@media (max-width: 1280px) { ... }
```

Характеристики страницы и окна браузера



orientation

ориентация окна браузера. Если высота окна браузера больше его ширины, то это портретная ориентация. Если наоборот, то это альбомная ориентация

```
@media (orientation: landscape) { ... }
```

ASPECT RATIOS



aspect-ratio, min-aspect-ratio, max-aspect-ratio

соотношение между шириной и высотой окна, минимальное и максимальное соотношение. Значение: два целых положительных числа, разделённых слэшем.

```
@media (aspect-ratio: 16 / 9) { ... }
```

Качество отображения

display-mode – проверяет, в каком режиме сайт или приложение запущено.

```
@media (display-mode: fullscreen) { ... }
```

scan – проверяет процесс отрисовки на девайсе.

```
@media (scan: interlace) { ... }
```

update – проверяет скорость обновления экрана.

```
@media (update: slow) { ... }
```

Цвет

color-gamut – примерный диапазон цветов, поддерживаемый браузером и устр-м вывода.

color, min-color, max-color – количество бит на цвет на устройстве вывода.

color-index, min/max-color-index – количество цветов, которое может отображаться устройством.

monochrome, min/max-monochrome – количество бит на цвет на монохромном устройстве вывода.

dynamic-range – комбинация уровня яркости, глубины цвета и контрастного соотношения для видео в браузере или устройстве вывода

Взаимодействие

hover – проверка, позволяет ли основное устройство наводить указатель на элементы.

any-hover – проверка, позволяет ли любое из устройств ввода наводить указатель на элементы.

pointer – проверка, является ли основное устройство ввода указателем, и насколько оно точное.

any-pointer – проверка, является ли любое из устройств ввода указателем, и насколько оно точное.

Хар-ки с префиксом video-

video-dynamic-range – комбинация уровня яркости, глубины цвета и контрастного соотношения для видео в браузере или устройстве вывода.

Значения:

- **standard** – если у устройства есть экран;
- **high** – устройство поддерживает высокую яркость, глубину и контраст.

```
@media (video-dynamic-range: high) { ... }
```


Пользовательские настройки

prefers-color-scheme – определяет, какую тему предпочитает пользователь — светлую или тёмную

prefers-reduced-motion – определяет, отключены ли анимации в системных настройках пользователя.

forced-colors – проверка на то, ограничивает ли браузер количество цветов в интерфейсе.

inverted-colors – проверка, инвертируются ли цвета браузером или ОС.

prefers-contrast – определяет, установлены ли настройки для увеличения или уменьшения контраста между фоном и текстом.

Скрипты

scripting – проверяет, включены ли скрипты.

Значения:

- **none** – скрипты отключены, недоступны;
- **initial-only** – скрипты доступны только в момент загрузки документа. После загрузки скрипты не работают;
- **enabled** – скрипты поддерживаются и доступны.



```
@media (scripting: initial-only) { ... }
```

Логические операторы

В медиавыражении может быть перечислено несколько условий через запятую. В таком случае стили применяются для любого из указанных условий. Часто в медиавыражениях может использоваться ключевое слово **and**, реже встречаются ключевые слова **not** и **only**.

```
@media (min-width: 320px) and (max-width: 640px) {  
  .link {  
    text-decoration: underline;  
  }  
}
```

```
@media only screen and (max-width: 600px) {  
  body {  
    background-color:  cornflowerblue;  
  }  
}
```

```
@media not screen and (height: 500px),  
print and (height: 700px) {  
  .text {  
    color:  tomato;  
  }  
}
```

Синтаксис диапазонов

В современных браузерах можно использовать синтаксис диапазонов для всех условий, где значение – число. Новые операторы `<`, `>`, `>=` и `<=` более интуитивно указывают, например, для каких размеров экрана применять правила из медиавыражения. На самом деле операторы не новые, каждый знаком с ними из курса школьной математики, но в медиавыражениях это новинка.

```
@media (min-width: 680px) and (max-width: 1280px) {  
  p {  
    color: red;  
  }  
}
```

=

```
@media (680px <=width <=1280px) {  
  p {  
    color: red;  
  }  
}
```

Подсказки

Для вёрстки от мобильных (mobile first) лучше использовать медиавыражения с **min-width** и располагать их в порядке увеличения ширины экрана;
для вёрстки от десктопа (desktop first) – **max-width**,
и располагать выражения в обратном порядке.

```
/* desktop first пример */
.text {
  color: navy;
  font-size: 18px;
}

@media (max-width: 1199px) {
  .text {
    color: darkblue;
  }
}

@media (max-width: 991px) {
  .text {
    color: blue;
  }
}
```

```
@media (max-width: 767px) {
  .text {
    color: royalblue;
  }
}

@media (max-width: 575px) {
  .text {
    color: dodgerblue;
    font-size: 16px;
  }
}

@media (max-width: 399px) {
  .text {
    color: lightblue;
    font-size: 14px;
  }
}
```

Медиавыражения можно вкладывать в медиавыражения и другие директивы

```
@media (min-width: 520px) {
  @media (max-width: 1080px) {
    .cell {
      background: peachpuff;
    }
  }
}
```

СПАСИБО ЗА ВНИМАНИЕ!

UI



UX



Usability



3 декабря 2025 года